

Docker 容器技术基础

王天青 | 2024年7月

日期	课程内容	作业布置
2024/7/1	1. 云原生与微服务架构实践	
2024/7/2	2. Docker 入门及实践	
2024/7/3	3. Docker 原理	第一次作业
2024/7/4	4. Kubernetes 入门及实践	
2024/7/5	5. Kubernetes 原理	
2024/7/6	6. Spring Boot 及 Spring Cloud 在 Kubernetes 上 up and running	第二次作业
2024/7/7	7. 基于 Kubernetes 实现 DevOps 流程	
2024/7/8	8. 课程总结	大作业

目录

CONTENTS

- 1 Docker 简介
- 2 Docker 入门
- 3 Docker 深入
- 4 容器技术标准

1 chapter

Docker简介

- ✓ 什么是 Docker
- ✓ 集装箱改变世界
- ✓ 集装箱对 Docker 的启发
- ✓ Docker 的特点
- ✓ Docker 的发展历程

➤ 概念

- Docker 容器、Docker 社区、Docker 公司。
- Docker 是 Docker 公司开源的一种最流行的容器实现方案，极大方便了应用服务部署。其容器技术已经成为容器行业的技术标准。
- Docker 可以将应用、配置和环境打包，形成一个独立的类似于 iOS APP 形式的「应用」。此「应用」可以直接分发到任意一个支持 Docker 的环境中，通过简单的命令即可运行。



➤ Docker 的用途

- **提供一次性的环境**

- 比如，本地测试他人的软件、持续集成的时候提供单元测试和构建的环境。

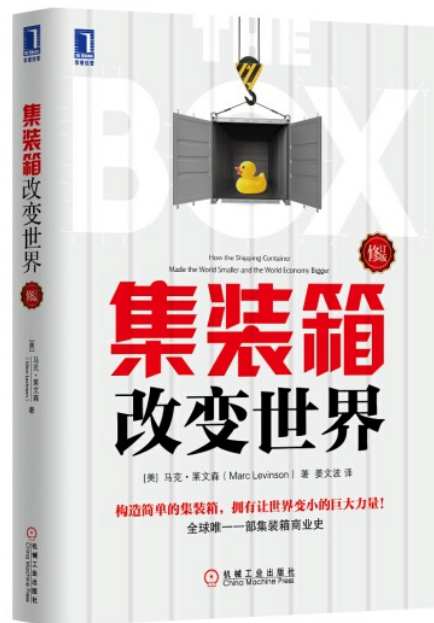
- **提供弹性的云服务**

- 因为 Docker 容器可以随开随关，很适合动态扩容和缩容。

- **组建微服务架构**

- 因为 Docker 容器非常地轻量，在一台机器通过运行多个容器就可以跑多个服务。

- 《集装箱改变世界》
 - “没有集装箱，就没有全球化” --- 《经济学家》
 - 集装箱何以彻底颠覆了全球运输业和世界经济？



➤ 集装箱式与传统货运方式对比

	散货运输	集装箱
货物装载时间	几天到一个星期，分拣、合并、装卸过程繁琐	几个小时
货物装卸时间占整个运输过程的比例	> 50%	< 10%
所需劳动力	大量码头工人	少数码头工人和起重机操作员
自动化程度	低	高
安全性	低，装卸时常有物品损毁和盗窃	高，货物被隔离在封闭的集装箱内
总体运输时间	货物装卸时间 + 实际运输时间	在总距离和运输工具的运输速度不变的前提下，可以节约一半以上的时间
成本	货物越多，成本越高，耗时越长	前期固定成本投入较高，随着集装箱规模运货量的增加，分摊到运送每个集装箱的成本较低，总成本几乎不变
货物运价	货运成本占了商品销售成本的9%-15%	运价降低到原来的 10%-20%，货运成本可以忽略不计

➤ 集装箱何以彻底颠覆了全球运输业和世界经济？

• 轻量标准化：**运输、装卸、存储**

- 集装箱最大的成功在于其产品的标准化以及由此建立的一整套运输体系。无论货物的体积、形状差异有多大，最终都被装载进集装箱里。由于要实现标准尺寸集装箱的运输，堆场、码头、起吊、船舶、汽车乃至公路桥梁、隧道等，都必须随着它在全球范围内的应用而逐渐加以标准化，形成影响国际贸易的全球物流系统。由此带来的是系统效率大幅度提升，运输费大幅度下降，地球上任何一个地方生产的产品都可以快速而低廉地运送到有需求的地方。



➤ 环境配置难题

- 用户计算机的环境都不相同，你怎么知道自家的软件，能在哪些机器跑起来？
- 用户必须保证两件事：操作系统的设置，各种库和组件的安装。只有它们都正确，软件才能运行
 - 举例来说，安装一个 Python 应用，计算机必须有 Python 引擎，还必须有各种依赖，可能还要配置环境变量。如果某些老旧的模块与当前环境不兼容，那就麻烦了。
 - 开发者常常会说：“在我的环境上还是可以的”。言下之意就是在当前的运行环境可能有什么“问题”，跟开发环境有什么不一样。
- 环境配置如此麻烦，换一台机器，就要重来一次，旷日费时。很多人想到，能不能从根本上解决问题，软件可以带环境安装，安装的时候把原始环境一模一样地复制过来

➤ 集装箱与容器类比

集装箱	Docker容器技术
发货商	应用的发布者，现实中多为应用的生产方，即开发者
客户	使用应用的互联网用户
货物	构成应用的代码、组件、依赖等
集装箱	Docker 容器
装卸货	应用的发布、撤销
码头工人	实际操作应用发布过程的人，现实中多为运维人员
散件装卸、运输方式	应用发布过程中逐个安装部署代码、组件、依赖、配置环境等
集装箱装卸、运输方式	把应用运行所需的外部环境，内部代码，组件，依赖打包进容器，应用发布以容器为单位
港口的码头、起重机、集装箱堆场	应用发布所需的基础设施与工具
轮船、轮船公司	容器运行平台，如可以运行容器的云计算平台

- Docker 对世界的改变（以回顾的视角来看）
 - 容器技术跟集装箱一样，给云计算行业带来了极大的技术变革，快速改变了公司和用户**创建、发布、运行**分布式应用的方式
 - Docker 使得容器化技术使用非常方便，极大地推进了容器行业的发展与容器技术标准化。
 - 容器镜像格式标准
 - 容器运行标准化
 - 就像标准化装卸集装箱货物一样极大方便了容器的存储、分发和使用。可以让开发者打包他们的应用及其依赖的环境到一个容器中，然后快速部署到任何一台Linux机器上。
 - Docker 的口号：**Build Once, Run Anywhere**



➤ Docker 的优势

- 环境一致性 Build Once, Run Anywhere
 - Docker容器镜像打包了应用服务运行环境
 - Docker容器基于开放式标准, 能够在所有主流 Linux 版本、Microsoft Windows 以及包括 VM、裸机服务器和云在内的任何基础设施上运行
- 资源独立与隔离
- 轻量化
 - 共享内核, 快速启动
 - 占用很少的计算、内存资源
 - 细粒度的资源管理方式
 - ✓ 支持0.5个CPU、200MB内存这种粒度的资源分配
 - 分层文件系统
 - ✓ 容器镜像通过分层文件系统组合而成, 共享相同的公共镜像层, 降低磁盘占用, 提高镜像分发速度

➤ 容器与虚拟机对比

特性	虚拟机	容器
隔离级别	系统级	进程级
隔离技术	Hypervisor 虚拟化整套硬件环境	Cgroups + Namespace 虚拟化进程运行环境
系统内核	独立内核	共享内核
性能开销	2~5%	< 0.5%
启动时间	分钟级	秒级
磁盘占用	GB级	MB级
部署密度	单机一般几十个	单机支持上千个容器

Hypervisor: 实现硬件资源虚拟化的软件技术/实现

➤ 容器与虚拟机类比

• 物理机

- 独栋别墅，一栋楼一户人家
- 独立地基、花园、家庭设施

• 虚拟机

- 商品房，一栋楼多套房、一套房一户人家
- 共享的地基、花园
- 独立的家庭设施：卫生间、厨房、宽带

• 容器

- 胶囊公寓，一个胶囊一位租户
- 共享地基、花园、卫生间、厨房、宽带
- 独立的床、柜子和照明



一栋楼一户人家，
独立地基，独立花园



一栋楼包含多套房，
一套房一户人家，
共享地基，共享花园，独
立卫生间、厨房和宽带



一套房隔成多个小隔间
(胶囊式公寓)，每个胶
囊住一位租户，共享地
基，共享花园，还共享
卫生间、厨房和宽带

➤ 容器的特点

- **容器是自包含的**

- 它打包了应用程序及其所有依赖，可以直接运行。以前应用程序的依赖管理一直是个大问题，即使像 RPM 、 ansible 等都能解决一部分问题，但并没有一个所有应用程序通用的标准机制，直到容器出现。

- **容器是可移植的**

- 可以在几乎任何地方以相同的方式运行。这就可以确保应用在开发环境、测试环境、生产环境等都有完全一样的运行环境。

- **容器是相互隔离的**

- 同一节点上运行的多个容器，不会互相影响。即一个容器中运行的应用程序，是访问不到其他容器的资源的（进程、网络、文件、用户等），除非配置为共享的资源。

- **容器是轻量级的**

- 体现在容器的秒级启动，并且占用资源很少。

➤ 容器与虚拟机使用上的差异

- ① 一个容器中只干一件事
- ② 容器的使用观念是我要启动一个 **APP**，而 VM 使用观念是我需要一个 **OS**
- ③ 容器通常是短暂性的，频繁更新的
- ④ 更新整个容器而非到容器中单独更新某个组件
- ⑤ 容器不需要热迁移
- ⑥ 容器镜像通常很小（相比于VM），分发十分快速
- ⑦ 创建容器非常方便，而创建VM相对复杂。
- ⑧ 容器与 host 共享内核
- ⑨ 容器中没有 **systemd**，1号进程就是应用进程本身

➤ 发展历程

- 2010年，一个做 PaaS 平台的小公司 **dotCloud** 在旧金山成立，该公司主要提供 PaaS 平台技术服务。
- 2013年，dotCloud 开源了其容器引擎 **Docker**，随后 Docker 容器技术大热，并在同年 dotCloud 公司更名为 Docker。
- 2014年，Docker 公司宣布把 PaaS 业务 dotCloud 出售给位于德国柏林的 PaaS 服务提供商 cloudControl，专门做容器技术。
- 2015年，Docker、CoreOS 和容器行业的其他领导厂商共同建立了 OCI (The Open Containers Initiative)，一个轻量级的开放式管理架构，来制定容器相关的规范、维护相关开源项目等。
- 2016 年，Docker 公司将 Docker 容器引擎的容器管理功能分拆出了 Containerd 并将其捐献给了社区。同年，Docker 发布 Swarm 与 Kubernetes 竞争容器编排市场。
- 2017年，容器编排领域市场竞争结果尘埃落定，Kubernetes 占据 **80%** 市场，Docker 官方开始拥抱 Kubernetes。

2 chapter

Docker入门

- ✓ Docker 安装
- ✓ Docker 基础概念
- ✓ Docker 镜像管理
- ✓ Docker 容器管理
- ✓ Docker 镜像构建
- ✓ Docker 常用配置

- Docker版本
 - 社区版 Docker CE （常用版本）
 - 企业版 Docker EE
- 参考官方文档安装Docker CE
 - Mac
 - Windows
 - Ubuntu
 - Debian
 - CentOS
 - Fedora
 - 其他 Linux 发行版

➤ 基础概念

- **宿主机 (Host)**

- Docker所在的物理机，是容器运行的系统环境。

- **镜像 (Image)**

- 镜像相当于一个程序模板，通过这个模板可以启动多个相似的容器。
- 镜像是静态的，就类似进程与程序中程序的概念。有镜像才能运行容器。

- **容器 (Container)**

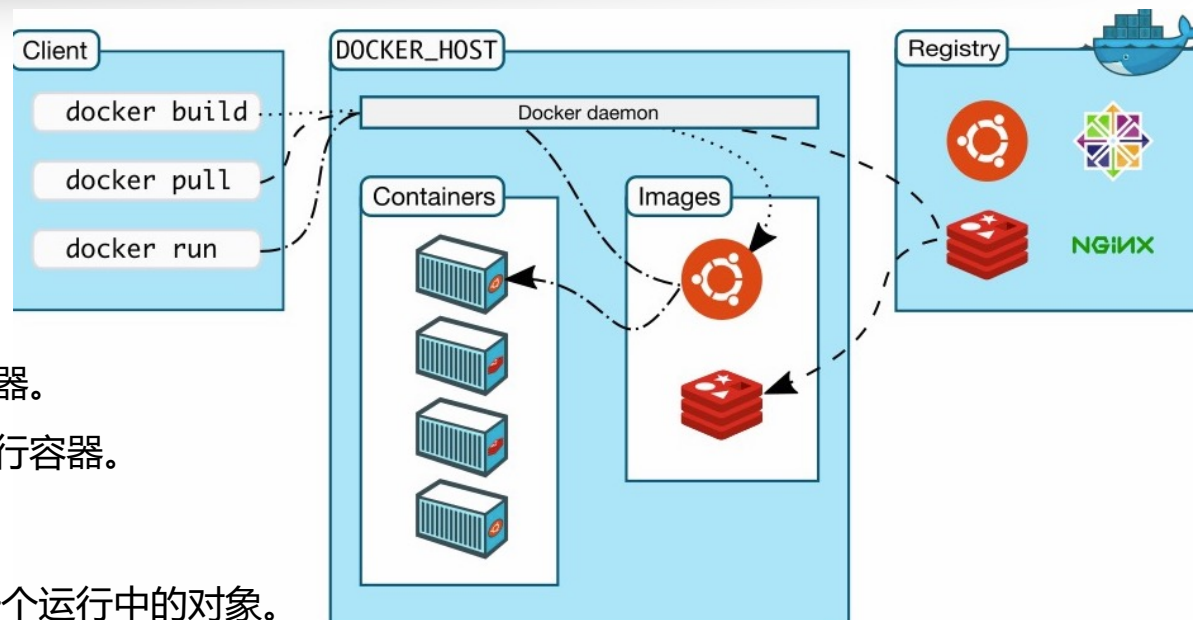
- 容器是docker运行的最小对象单元，是通过镜像实例化出来的一个运行中的对象。
- 容器是动态的，就类似进程与程序中进程的概念。容器可以导出、保存为镜像。

- **仓库 (Registry)**

- 镜像仓库是集中式的管理、存储镜像的仓库，类似Git管理代码的仓库一样，可以管理多版本的镜像。
- 镜像仓库以HTTP 的方式对外提供服务。可以通过docker命令从registry中pull到镜像，或者将镜像push到registry中。

➤ 关系总结

- 从**镜像仓库**中拉取到**容器镜像**，使用容器镜像在**宿主机**上运行**容器**。



➤ 镜像的表示

- 完整的镜像名由<repository>:<tag>两部分组成

- repository 由远端仓库地址、项目、 镜像名 组成（参考右图）
- tag 用来标记镜像的版本，不指定则默认为latest

repository tag
registry.transwarp.io/demo/centos:latest
远端仓库地址 项目/镜像名 版本

➤ 远端镜像仓库

- 解析远程仓库地址
 - docker会根据上述镜像表示的规则，从完整镜像名中的repository字段解析出远程仓库（registry）地址，访问此远程仓库地址（ registry ）来获取或上传镜像。
- 相关规范标准
 - [镜像格式： Registry image manifests](#)
 - [镜像仓库认证： Token Authentication Specification](#)

➤ 列出本地镜像

```
docker images [OPTIONS] [REPOSITORY[:TAG]]
```

```
$ sudo docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
busybox	latest	beae173ccac6	3 weeks ago	1.24MB
nginx	latest	605c77e624dd	3 weeks ago	141MB
kubernetes/ks-installer	v3.2.1	7f833c28ad87	4 weeks ago	383MB
rancher/rancher	v2.6.3	f9e320b7e19c	5 weeks ago	1.16GB
debian	buster	8a94f77c4ac3	5 weeks ago	114MB
kubernetes/ks-controller-manager	v3.2.1	ea90d211235f	5 weeks ago	168MB

➤ 拉取镜像到本地

```
docker pull [OPTIONS] NAME[:TAG|@DIGEST]
```

```
$ sudo docker pull busybox
Using default tag: latest
latest: Pulling from library/busybox
Digest: sha256:5acba83a746c7608ed544dc1533b87c737a0b0fb730301639a0179f9344b1678
Status: Downloaded newer image for busybox:latest
docker.io/library/busybox:latest
```

➤ 给镜像打 tag

```
docker tag SOURCE_IMAGE[:TAG] TARGET_IMAGE[:TAG]
```

```
$ sudo docker tag busybox:latest 172.16.1.99/tmp/busybox:latest
```

```
$ sudo images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
172.16.1.99/tmp/busybox	latest	beae173ccac6	3 weeks ago	1.24MB
busybox	latest	beae173ccac6	3 weeks ago	1.24MB

➤ 上传本地镜像

```
docker login [OPTIONS] [SERVER]
docker push [OPTIONS] NAME[:TAG|@DIGEST]
```

```
$ sudo docker login -u tmp 172.16.1.99
```

Password:

WARNING! Your password will be stored unencrypted in /root/.docker/config.json.

Configure a credential helper to remove this warning. See

<https://docs.docker.com/engine/reference/commandline/login/#credentials-store>

Login Succeeded

```
$ sudo docker push 172.16.1.99/tmp/busybox:latest
```

The push refers to repository [172.16.1.99/tmp/busybox]

01fd6df81c8e: Pushed

latest: digest: sha256:62ffc2ed7554e4c6d360bce40bbcf196573dd27c4ce080641a2c59867e732dee size: 527

➤ 删除本地镜像

```
docker rmi [OPTIONS] IMAGE [IMAGE...]
```

```
$ sudo docker rmi 172.16.1.99/tmp/busybox:latest
```

```
Untagged: 172.16.1.99/tmp/busybox:latest
```

```
Untagged:
```

```
172.16.1.99/tmp/busybox@sha256:62ffc2ed7554e4c6d360bce40bbcf196573dd27c4ce080641a2c59867e732  
dee
```

➤ 本地镜像

- docker images : 列出本地镜像。
- docker rmi : 删除本地一个或多个镜像。
- docker tag : 标记本地镜像, 将其归入某一仓库。
- docker build : 根据 Dockerfile 构建镜像。
- docker commit: 将容器commit为镜像
- docker history : 查看指定镜像的创建历史。
- docker save : 将指定镜像保存成 tar 归档文件。
- docker load : 导入使用 docker save 命令导出的镜像。
- docker import : 从归档文件中创建镜像。

➤ 镜像仓库

- docker login : 登陆到一个Docker镜像仓库, 如果未指定镜像仓库地址, 默认为官方仓库 Docker Hub
- docker logout : 登出一个Docker镜像仓库, 如果未指定镜像仓库地址, 默认为官方仓库 Docker Hub
- docker pull : 从镜像仓库中拉取或者更新指定镜像
- docker push : 将本地的镜像上传到镜像仓库, 要先登陆到镜像仓库
- docker search : 从Docker镜像仓库中查找镜像

➤ 管理Docker镜像跟管理代码一样需要多版本控制，镜像管理的方式参考了Git

Docker	Git
镜像仓库 (docker-registry、harbor)	代码仓库 (github、gitlab)
用户 (user)	用户 (user)
仓库 (repository)	仓库 (repository)
镜像 (image)	分支 (branch)
镜像层 (layers)	代码提交 (commits)
docker pull	git pull
docker push	git push
docker commit	git commit
docker diff	git diff
docker history	git log
镜像表示: registry.transwarp.io/demo/centos:latest	代码仓库表示: https://gitlab.transwarp.io/demo/centos.git

➤ 运行容器

```
docker run [OPTIONS] IMAGE [COMMAND] [ARG...]
```

```
$ sudo docker run -it centos:latest /bin/bash
Unable to find image 'centos:latest' locally
latest: Pulling from library/centos
Digest: sha256:a27fd8080b517143cbbbab9dfb7c8571c40d67d534bbdee55bd6c473f432b177
Status: Downloaded newer image for centos:latest
root@3b3d676c0d78 /]#
[root@3b3d676c0d78 /]# ls
bin dev etc home lib lib64 lost+found media mnt opt proc root run sbin srv sys tmp usr var
```

➤ 运行容器，并把 host 中目录挂载到容器中

```
$sudo docker run -ti -v /tmp:/tmp centos:latest bash
[root@1018b7b12176 /]# ls /tmp/
Temp-17aeecb-a669-4374-86e4-0c39ac54c7ef    systemd-private-5cb2bc32481a4fcda85256fc8e68b478-
colord.service-RCnDZh    tmux-0
```

➤ 查看运行的容器

```
docker ps [OPTIONS]
```

```
$ sudo docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS   NAMES
17f9ed6b58f6   nginx     "/docker-entrypoint..." 1 second ago   Up Less than a
second 80/tcp    dazzling_austin
0d2701ad18be   nginx     "/docker-entrypoint..." 16 minutes ago Up 2
minutes 80/tcp   serene_hopper
```

➤ 查看所有容器（包含停止的）

```
$ sudo docker ps -a
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS   NAMES
17f9ed6b58f6   nginx     "/docker-entrypoint..." 2 minutes ago   Up 2
minutes      80/tcp    dazzling_austin
1018b7b12176   centos:latest "bash"                  29 minutes ago Exited (130) 28 minutes
ago          pensive_mestorf
```

➤ 在运行的容器中执行命令

```
docker exec [OPTIONS] CONTAINER COMMAND [ARG...]
```

➤ 进入已在运行的容器中（即在此容器中运行一个bash）

```
$ sudo docker ps
7ae4495190a0  centos:latest          "tail -f /dev/null"    10 seconds ago  Up 9
seconds      ecstatic_kilby

$ sudo docker exec -ti ecstatic_kilby bash
[root@7ae4495190a0 /]# ls
bin dev etc home lib lib64 lost+found media mnt opt proc root run sbin srv sys tmp usr var
[root@7ae4495190a0 /]#
```

➤ 查看容器日志

```
docker logs [OPTIONS] CONTAINER
```

```
$ sudo docker run -d nginx
a17f3ee601cf9d1933e5712e8e6c97d8c11aef4d31bddf3646fc34cff35fb4ca
$ sudo docker logs a17f3ee60
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2022/01/26 06:09:22 [notice] 1#1: using the "epoll" event method
2022/01/26 06:09:22 [notice] 1#1: nginx/1.21.5
2022/01/26 06:09:22 [notice] 1#1: built by gcc 10.2.1 20210110 (Debian 10.2.1-6)
2022/01/26 06:09:22 [notice] 1#1: start worker processes
```

➤ 删除容器

```
docker rm [OPTIONS] CONTAINER [CONTAINER...]
```

```
$ sudo docker rm pensive_kowalevski
Error response from daemon: You cannot remove a running container
a17f3ee601cf9d1933e5712e8e6c97d8c11aef4d31bddf3646fc34cff35fb4ca. Stop the container before
attempting removal or force remove
```

```
$ sudo docker rm pensive_kowalevski -f
pensive_kowalevski
```

➤ 杀死容器

```
docker kill [OPTIONS] CONTAINER [CONTAINER...]
```

```
$ sudo docker kill ecstatic_kilby
ecstatic_kilby
```

➤ 停止在运行的容器

```
docker stop [OPTIONS] CONTAINER [CONTAINER...]
```

```
sudo docker stop serene_hopper
serene_hopper
$sudo docker ps -a |grep serene_hopper
0d2701ad18be  nginx  "/docker-entrypoint..." 7 minutes ago  Exited (0) 10 seconds ago  serene_hopper
```

➤ 启动一个被停止的容器

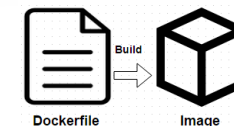
```
docker start [OPTIONS] CONTAINER [CONTAINER...]
```

```
$sudo docker start serene_hopper
serene_hopper

$sudo docker ps |grep serene_hopper
0d2701ad18be  nginx  "/docker-entrypoint..." 13 minutes ago  Up 7 seconds  80/tcp  serene_hopper
```


➤ Dockerfile

- Dockerfile 是描述镜像如何构建的「脚本」
 - Dockerfile 中的每一条指令都会创建一个镜像层，每多一条指令便会增加一层镜像的大小。
 - 容器环境的构建过程都以一条条指令的方式清晰地记录在Dockerfile，开发者像开发代码一样编写Dockerfile来构建容器应用运行环境。
- Dockerfile 示例



```
$ cat Dockerfile
FROM centos:8
RUN dd if=/dev/zero of=/tmp/file bs=1M count=1
```

➤ 使用 Dockerfile 构建镜像

```
docker build [OPTIONS] PATH | URL | -
```

```
$ ls
```

```
Dockerfile
```

```
$ sudo docker build -t test-image-based-on-centos:8 .
```

```
Sending build context to Docker daemon 3.072kB
```

```
Step 1/2 : FROM centos:8
```

```
---> 5d0da3dc9764
```

```
Step 2/2 : RUN dd if=/dev/zero of=/tmp/file bs=1M count=1
```

```
---> Running in 24a480a3c030
```

```
Removing intermediate container 24a480a3c030
```

```
---> a451754fb42f
```

```
Successfully built a451754fb42f
```

```
Successfully tagged test-image-based-on-centos:8
```

➤ Dockerfile 关键字详解

- **FROM**

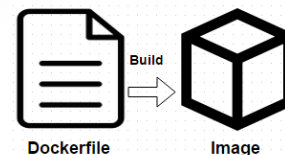
- FROM指令 指定构建当前镜像的基础镜像。
- Dockerfile中构建镜像必须从FROM一个Base镜像开始，如果不依赖任何基础镜像，可以 “FROM scratch” ，表示一个空镜像作为基础镜像。

- **RUN**

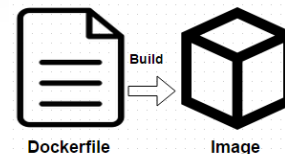
- 执行一条命令，默认是以 /bin/sh -c 执行。例如：RUN apt-get update -y && apt-get install -y nginx

- 示例

```
FROM centos:8  
RUN yum install -y nginx
```



➤ Dockerfile关键字详解



• COPY

- COPY 指令将从构建上下文目录中 <源路径> 的文件/目录复制到新的一层的镜像内的 <目标路径> 位置。

• ADD

- ADD 指令和 COPY 的格式和性质基本一致。但是在 COPY 基础上增加了一些功能。
 - ✓ <源路径> 可以是一个 URL
 - ✓ <源路径> 是一个 tar 压缩文件并且压缩格式为 gzip, bzip2 和 xz 的情况下, ADD 指令将会自动解压缩这个压缩文件到 <目标路径> 中。

• 示例

```
FROM scratch
ADD ubuntu-xenial-core-cloudimg-amd64-root.tar.gz /
COPY test.html /var/www/html/
```

➤ Dockerfile关键字详解

• CMD

- CMD指令定义容器的启动命令，例如：CMD ["nginx", "-g", "daemon off;"]

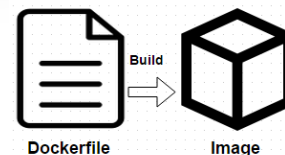
• ENTRYPOINT

- ENTRYPOINT与CMD类似，都是用于指定容器的执行入口，好处是可以将容器当命令使用

• 示例

```
FROM centos:8
RUN yum install -y curl
ENTRYPOINT ["curl"]
```

```
$ sudo docker run -t curl-in-container:latest --help
Usage: curl [options...] <url>
  --abstract-unix-socket <path> Connect via abstract Unix domain socket
  --anyauth             Pick any authentication method
-a, --append           Append to target file when uploading
  --basic              Use HTTP Basic Authentication
  --cacert <file>      CA certificate to verify peer against
```



➤ Dockerfile关键字详解

- **ENV**

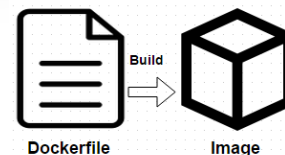
- ENV指令用于设置容器内环境变量。
- 使用格式: ENV <key> <value> 或 ENV <key1>=<value1> <key2>=<value2>....

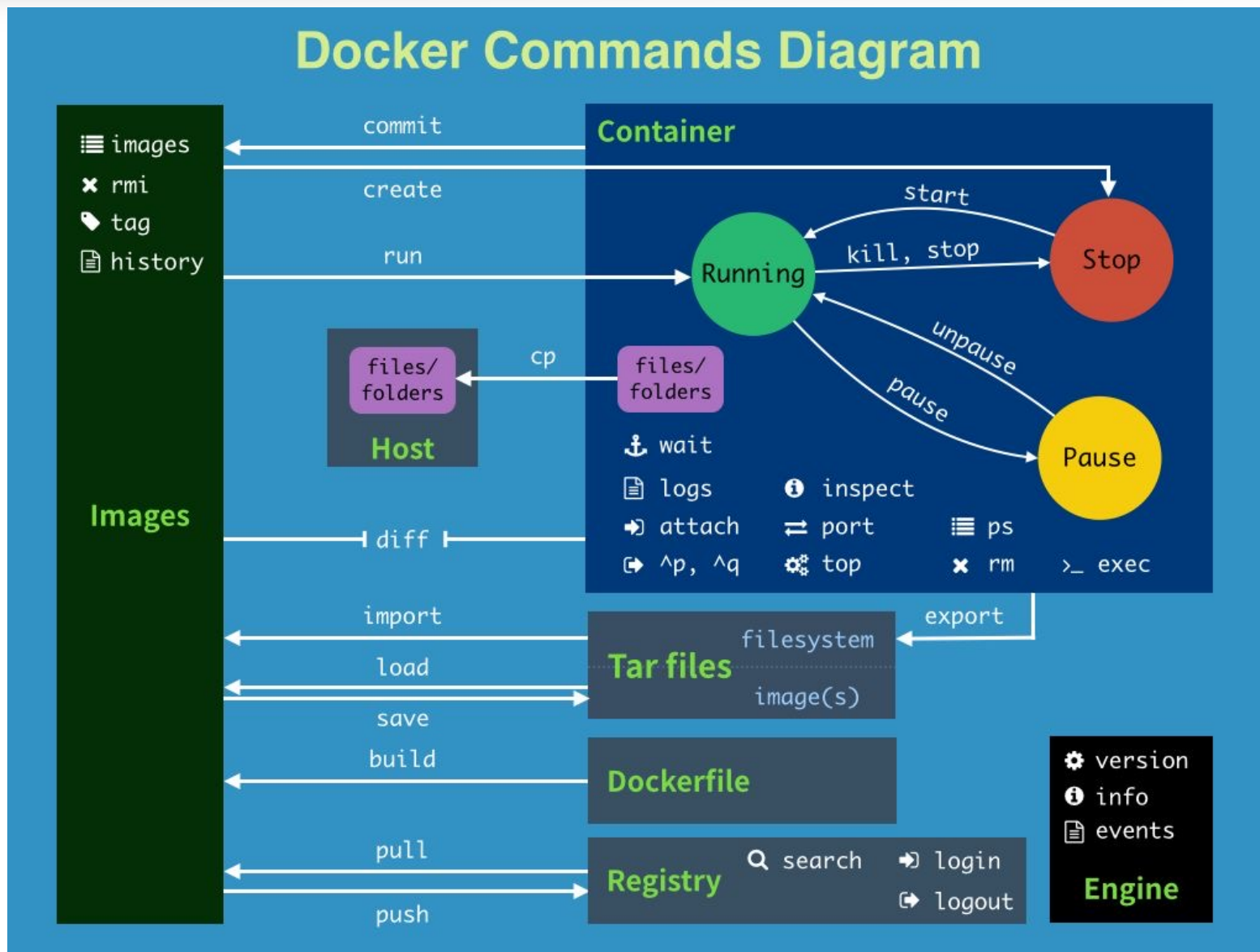
- **WORKDIR**

- WORKDIR 指令可以来指定工作目录（或者称为当前目录），以后各层的当前目录就被改为指定的目录，如该目录不存在，WORKDIR 会自动创建。
- 格式为 WORKDIR <工作目录路径>。

- **EXPOSE**

- EXPOSE 指令是声明容器运行时提供服务的端口。





Container Management

<code>docker ps</code>	List the running containers.
<code>docker ps -a</code>	List all the containers, both running and stopped.
<code>docker create [image]</code>	Create a container without starting it.
<code>docker create -it [image]</code>	Create an interactive container with pseudo-TTY.
<code>docker rename [container] [new-name]</code>	Rename a container.
<code>docker rm [container]</code>	Remove a stopped container.
<code>docker rm -f [container]</code>	Force remove a container, even if it is running.
<code>docker logs [container]</code>	View logs for a running container.
<code>docker logs -f --until=[interval] [container]</code>	Retrieve logs before a specific point in time.
<code>docker events [container]</code>	View real time events for a container.
<code>docker update [container]</code>	Update the configuration of a container.
<code>docker port [container]</code>	Show port mapping for a container.
<code>docker top [container]</code>	Show running processes in a container.
<code>docker stats [container]</code>	Show live resource usage statistics for a container.
<code>docker diff [container]</code>	Show changes to files or directories on the filesystem.
<code>docker cp [file-path] CONTAINER:[path]</code>	Copy a local file to a directory in a container.



Networking

<code>docker network ls</code>	View available networks.
<code>docker network rm [network]</code>	Remove a network.
<code>docker network inspect [network]</code>	Show information about a network.
<code>docker network connect [network] [container]</code>	Connect a container to a network.
<code>docker network disconnect [network] [container]</code>	Disconnect a container from a network.

General Management

<code>docker login</code>	Log in to a Docker registry.
<code>docker logout</code>	Log out of a Docker registry.
<code>docker inspect [object]</code>	Show low-level information about an object.
<code>docker version</code>	Show the version of the local Docker installation.
<code>docker info</code>	Display information about the system.
<code>docker system prune</code>	Remove unused images, containers, and networks.



Running a Container

<code>docker run [image] [command]</code>	Run a command in a container based on an image.
<code>docker run --name [container-name] [image]</code>	Create, start, and name a container.
<code>docker run -p [host]:[container-port] [image]</code>	Map a host port to a container port.
<code>docker run --rm [image]</code>	Run a container and remove it after it stops.
<code>docker run -d [image]</code>	Run a detached (background) container.
<code>docker run -it [image]</code>	Run an interactive process, e.g., a shell, in a container.
<code>docker start [container]</code>	Start a container.
<code>docker stop [container]</code>	Stop a container.
<code>docker restart [container]</code>	Stop a container and start it again.
<code>docker pause [container]</code>	Pause processes in a running container.
<code>docker unpause [container]</code>	Unpause processes in a running container.
<code>docker wait [container]</code>	Block input until the container stops.
<code>docker kill [container]</code>	Send a SIGKILL signal to stop a container.
<code>docker attach [container]</code>	Attach local standard input, output and error.
<code>docker exec -it [container] [shell]</code>	Run a shell inside a running container.

Image Management

<code>docker build [dockerfile-path]</code>	Create an image from a Dockerfile.
<code>docker build .</code>	Build an image using the files from the current path.
<code>docker build -t [name]:[tag] [location]</code>	Create an image from a Dockerfile and tag it.
<code>docker build -f [file]</code>	Specify a file to build from.
<code>docker pull [image]</code>	Pull an image from a registry.
<code>docker push [image]</code>	Push an image to a registry.
<code>docker import [url/file]</code>	Create an image from a tarball.
<code>docker commit [container] [new-image]</code>	Create an image from a container.
<code>docker tag [image] [image]:[tag]</code>	Tag an image.
<code>docker images</code>	Show all locally stored top level images.
<code>docker history [image]</code>	Show history for an image.
<code>docker rmi [image]</code>	Remove an image.
<code>docker load --image [tar-file]</code>	Load an image from a tar archive file.
<code>docker save [image] > [tar-file]</code>	Save an image to a tar archive file.
<code>docker search [query]</code>	Search Docker Hub for images.
<code>docker image prune</code>	Remove unused images.



Plugin Management

<code>docker plugin enable [plugin]</code>	Enable a Docker plugin.
<code>docker plugin disable [plugin]</code>	Disable a Docker plugin.
<code>docker plugin create [plugin] [path-to-data]</code>	Create a plugin from config.json and rootfs.
<code>docker plugin inspect [plugin]</code>	View details about a plugin.
<code>docker plugin rm [plugin]</code>	Remove a plugin.

- Docker Daemon 配置文件
 - /etc/docker/daemon.json
- 配置 registry mirrors
 - 在 daemon.json 中配置 registry-mirrors 来指定 registry 的 mirrors
 - 例如: {"registry-mirrors": ["https://y7oivp0s.mirror.aliyuncs.com"]}
- 配置开启允许访问不安全的 registry
 - 在 daemon.json 中配置 insecure-registries 来指定允许访问的不安全（非https）的 registry

```
$ sudo cat >/etc/docker/daemon.json <<EOF
> {
>   "registry-mirrors": ["https://y7oivp0s.mirror.aliyuncs.com"],
>   "insecure-registries": ["172.16.1.99"]
> }
> EOF
$ sudo systemctl restart docker
```

- Docker systemd service 配置文件
 - /usr/lib/systemd/system/docker.service
 - /etc/systemd/system/docker.service.d/
- 配置 docker 服务的 http 代理

```
$ sudo cat > /etc/systemd/system/docker.service.d/http-proxy.conf<<EOF
> [Service]
> Environment="HTTP_PROXY=http://172.16.2.14:7777"
> Environment="HTTPS_PROXY=http://172.16.2.14:7777"
> Environment="NO_PROXY=localhost,127.0.0.1,172.22.28.75,https://y7oivp0s.mirror.aliyuncs.com"
EOF
$ sudo systemctl daemon-reload
$ sudo systemctl restart docker
```

3 chapter

Docker深入

- ✓ Docker 容器技术背景
- ✓ Docker 组成架构
- ✓ Docker 容器技术基石
- ✓ 深入理解 Docker

➤ Docker 容器技术背景

- 行业痛点

- 容器技术的兴起源于 PaaS 技术的普及。在以往的 PaaS 项目中，用户会把**应用的可执行文件和启动脚本打进一个压缩包**内，然后大规模部署应用到集群里。但是这个打包的工作却一波三折，费尽心机。用户需要为每种语言、每种框架、每个版本的应用维护一个打好的包，并且要做很多修改和配置工作才能在 PaaS 里运行起来。

- Docker 方案

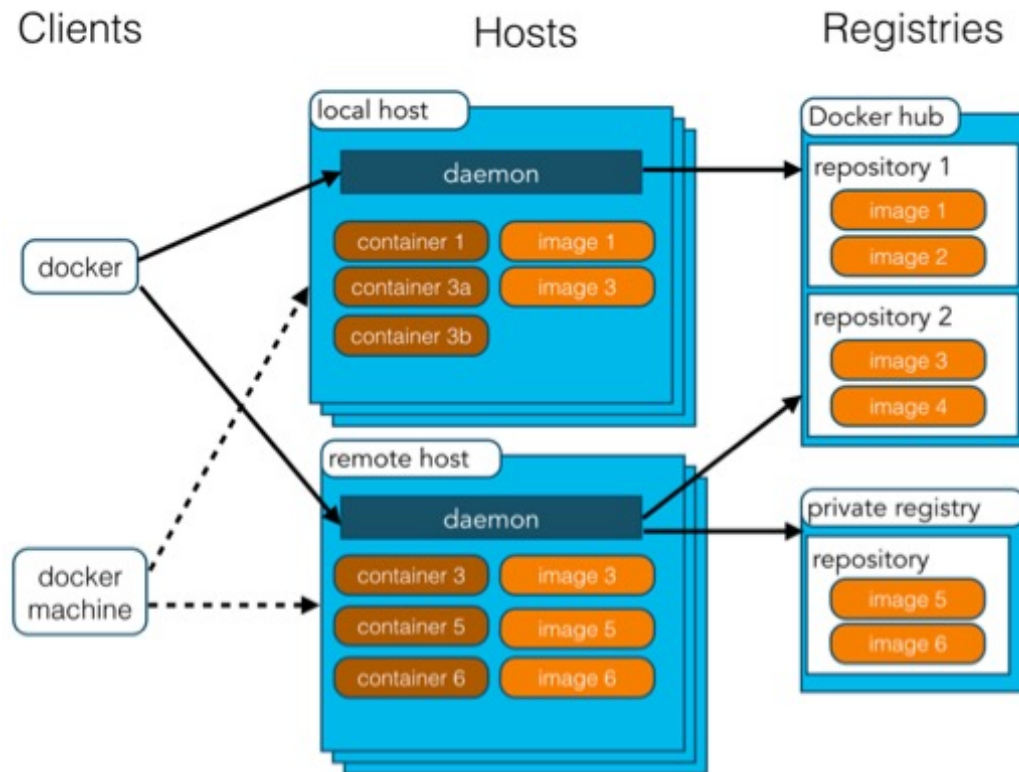
- Docker 镜像解决了打包这个根本性问题。**所谓 Docker 镜像（不是 Docker 容器），其实就是一个压缩包。**但是这个压缩包里的内容，却不只应用的可执行文件和启动脚本。大多数 Docker 镜像是直接由一个完整操作系统的所有文件和目录构成的，所以这个压缩包里的内容跟你本地开发和测试环境用的操作系统是完全一样的。

- Docker 优势

- Docker **这种打包机制直接打包了应用运行所需要的整个操作系统，所需要的所有依赖**，从而保证了本地环境和云端环境的高度一致，避免了用户通过“试错”来匹配两种不同运行环境之间差异的痛苦过程。

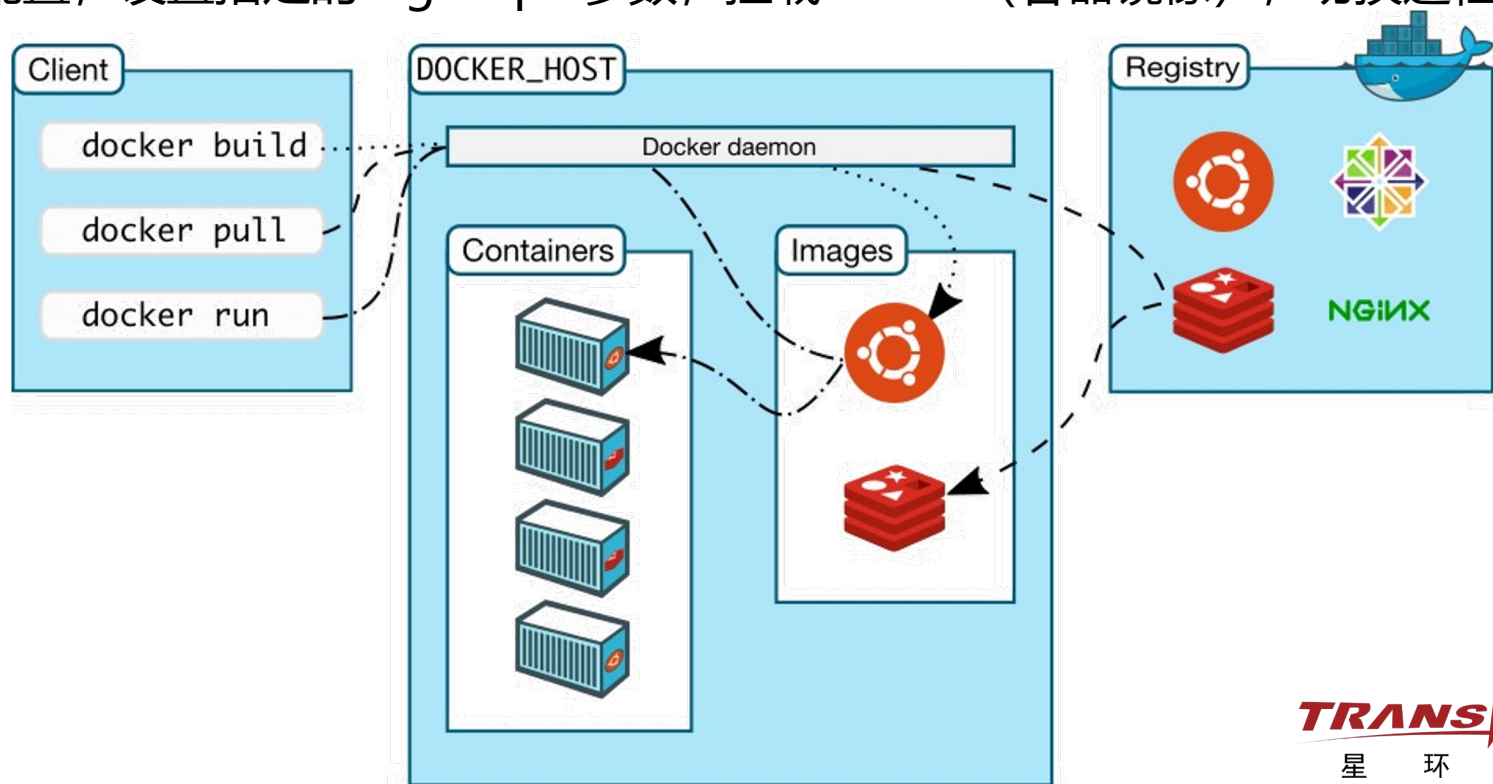
➤ Docker架构概览

- Docker 采用 C/S 架构
 - Client 通过接口与 Server 进程通信实现容器的构建，运行和发布。
 - Docker 使用 C/S 架构，Client 通过接口与 Server 进程通信实现容器的构建，运行和发布。
- Docker由五个部分组成
 - **Docker Client 客户端**，负责向服务端（Docker Daemon 守护进程）发起请求
 - **Docker Daemon 守护进程**，负责从 Docker Registry 下载 Docker 镜像和通过 Docker 镜像 启动 Docker 容器
 - **Docker Registry 仓库**，负责存储 Docker 镜像
 - **Docker Container 容器**
 - **Docker Image 镜像**



➤ Docker 运行一个容器的大致流程

- ① Docker Client 执行 docker run, Docker Daemon 收到来自 client 的请求;
- ② Docker Daemon 优先查找本地镜像, 如果本地没有, Docker daemon 会从远端镜像仓库拉取所需镜像;
- ③ 启动 Linux Namespace 配置, 设置指定的 Cgroups 参数, 挂载 rootfs (容器镜像), 切换进程的根目录;
- ④ 容器运行。



➤ Docker Daemon 的功能由 4 个组件构成

- **dockerd**

- 对外 API 服务
- 镜像管理

- **containerd**

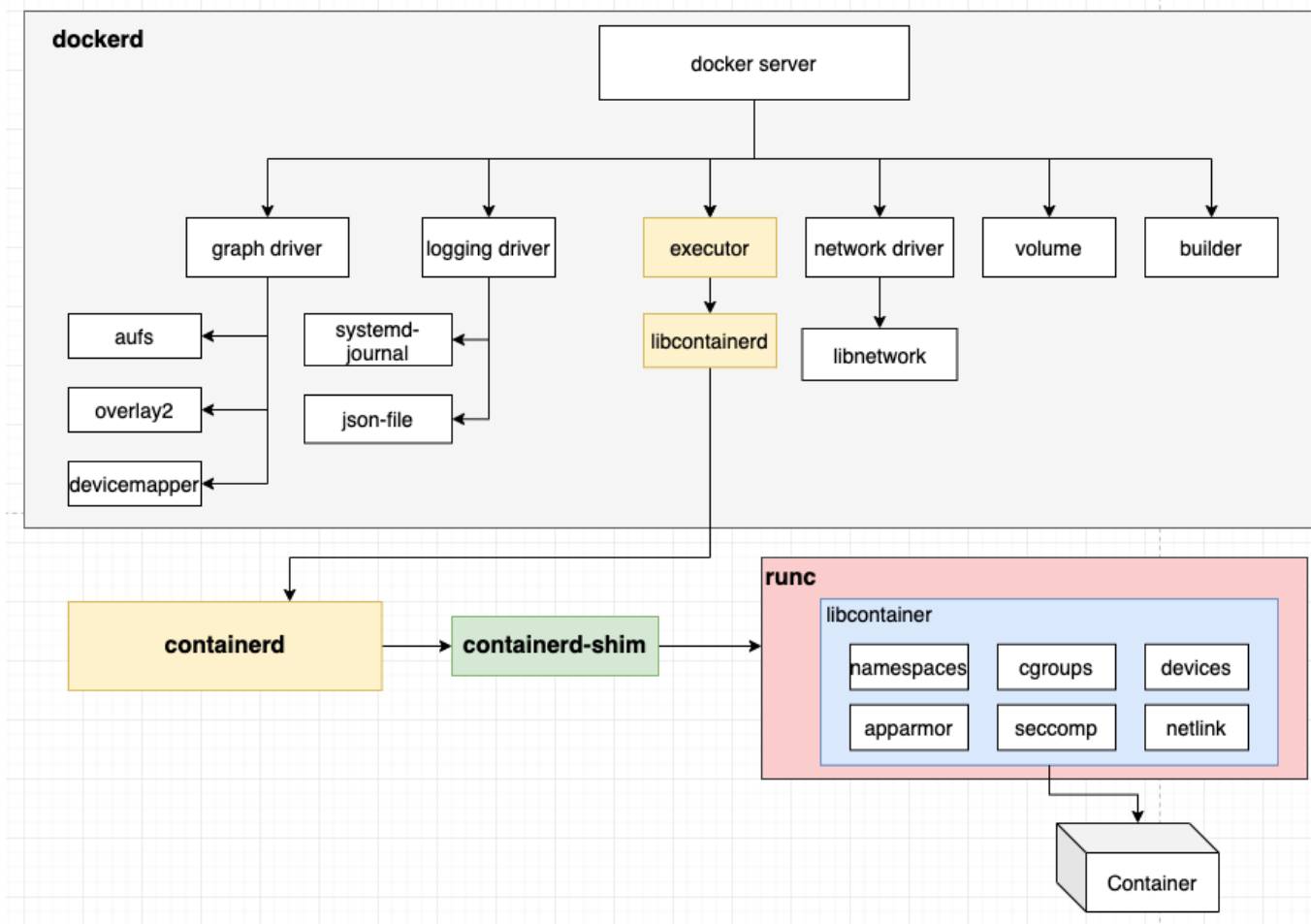
- 容器生命周期管理

- **containerd-shim**

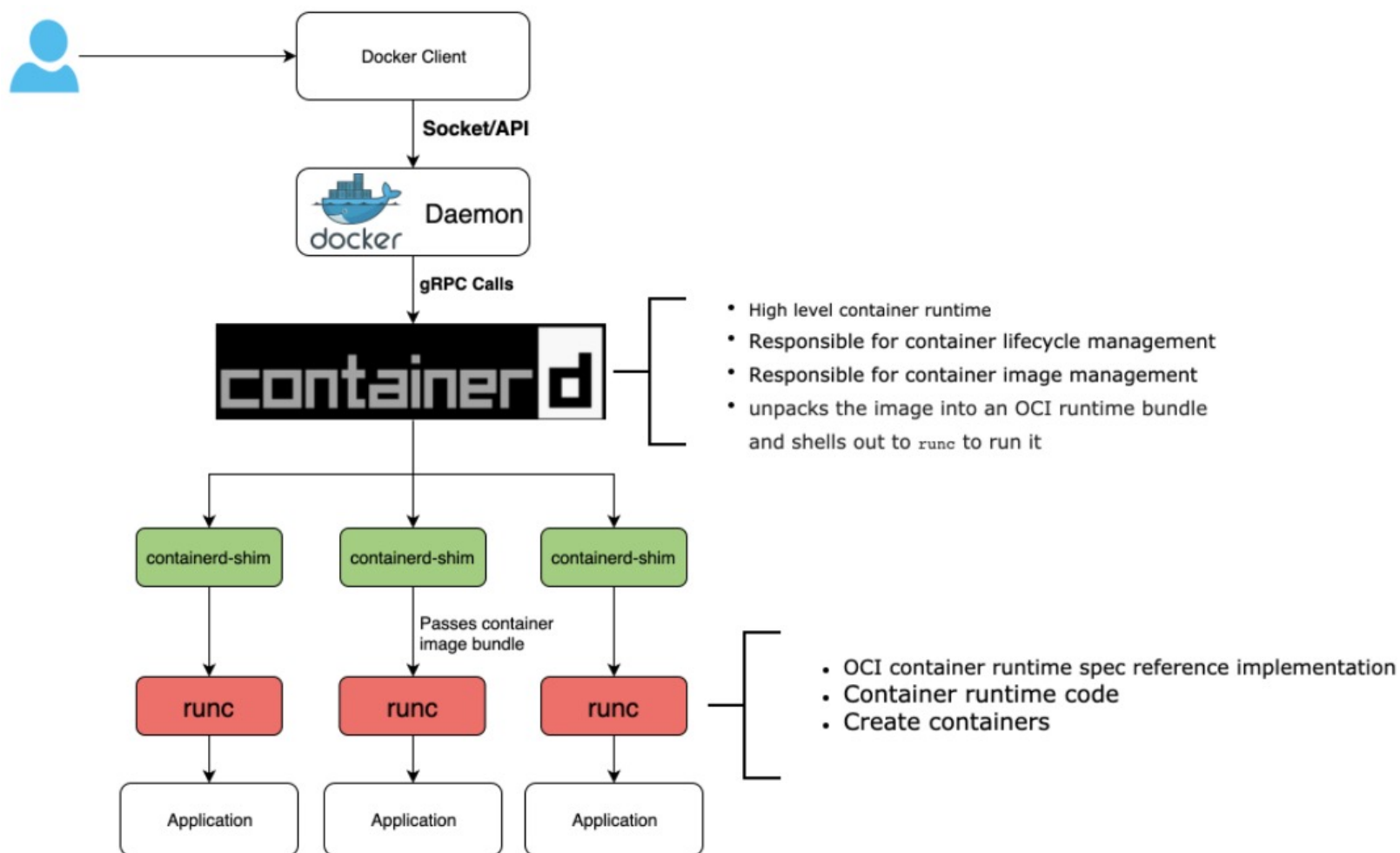
- 容器在 Host 上的父进程
- 用于将容器生命周期与 containerd 进程解绑

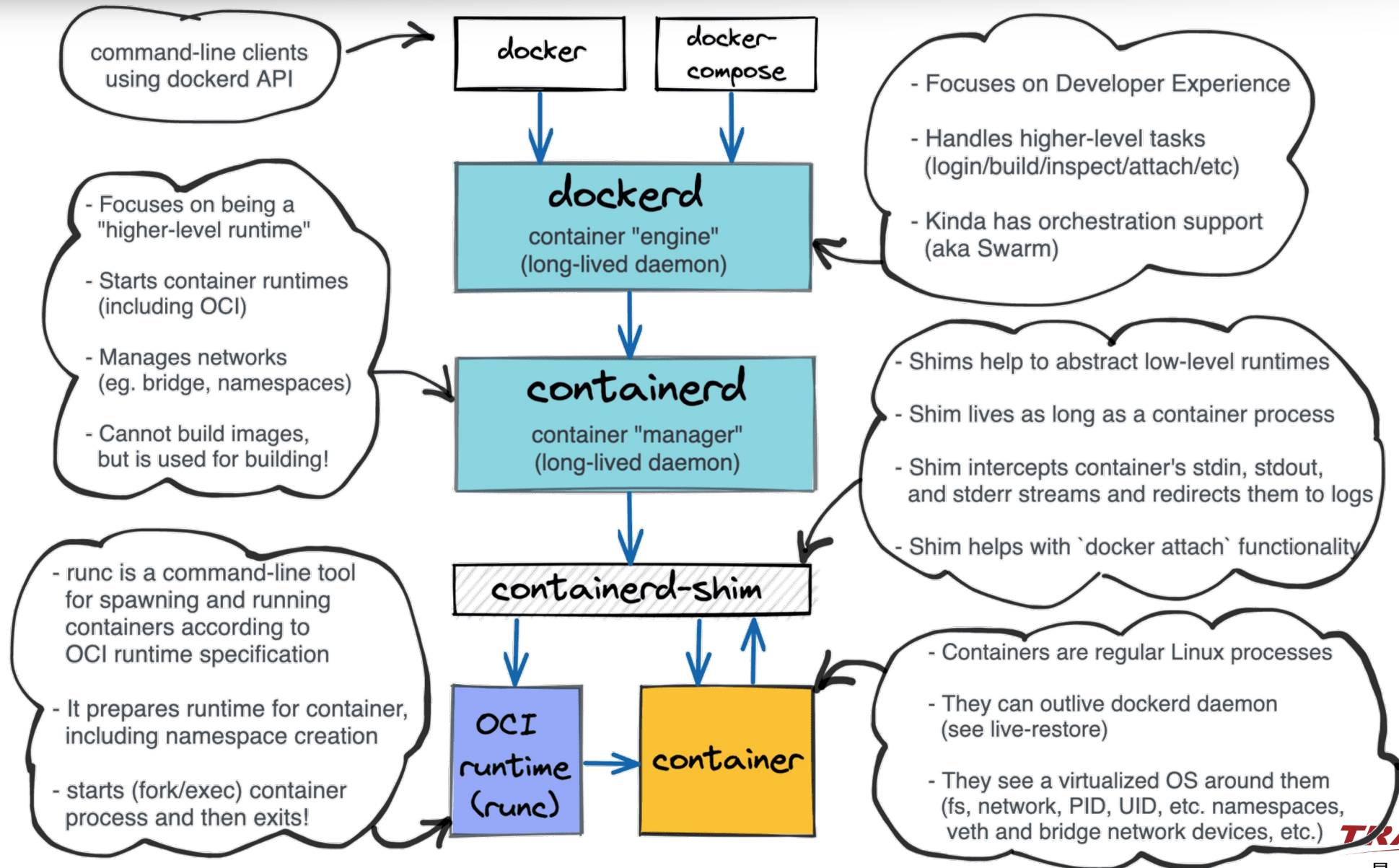
- **runc**

- 容器运行时 (runtime)
- 直接创建、删除、启动、停止容器等



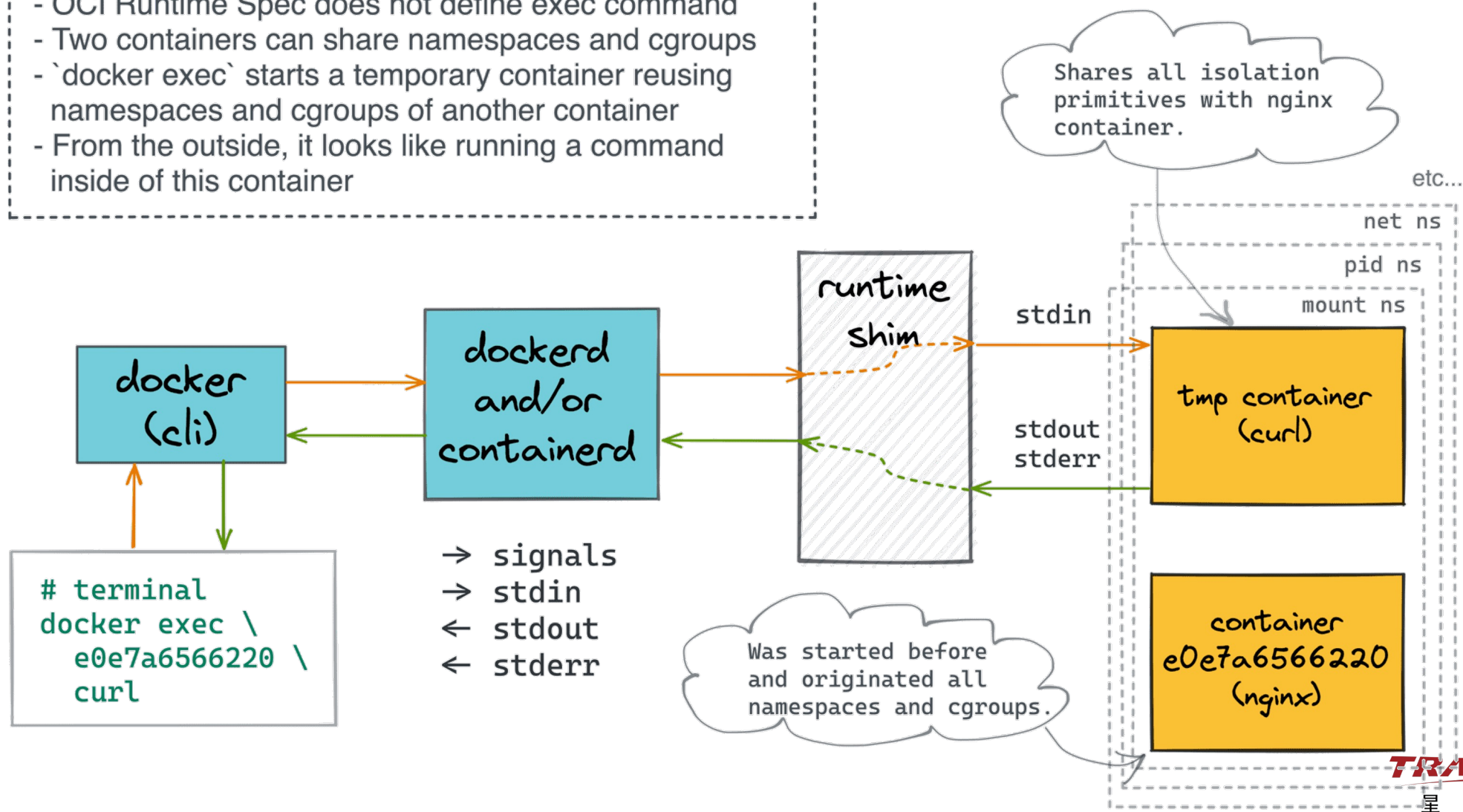
➤ Docker 组件细分后架构

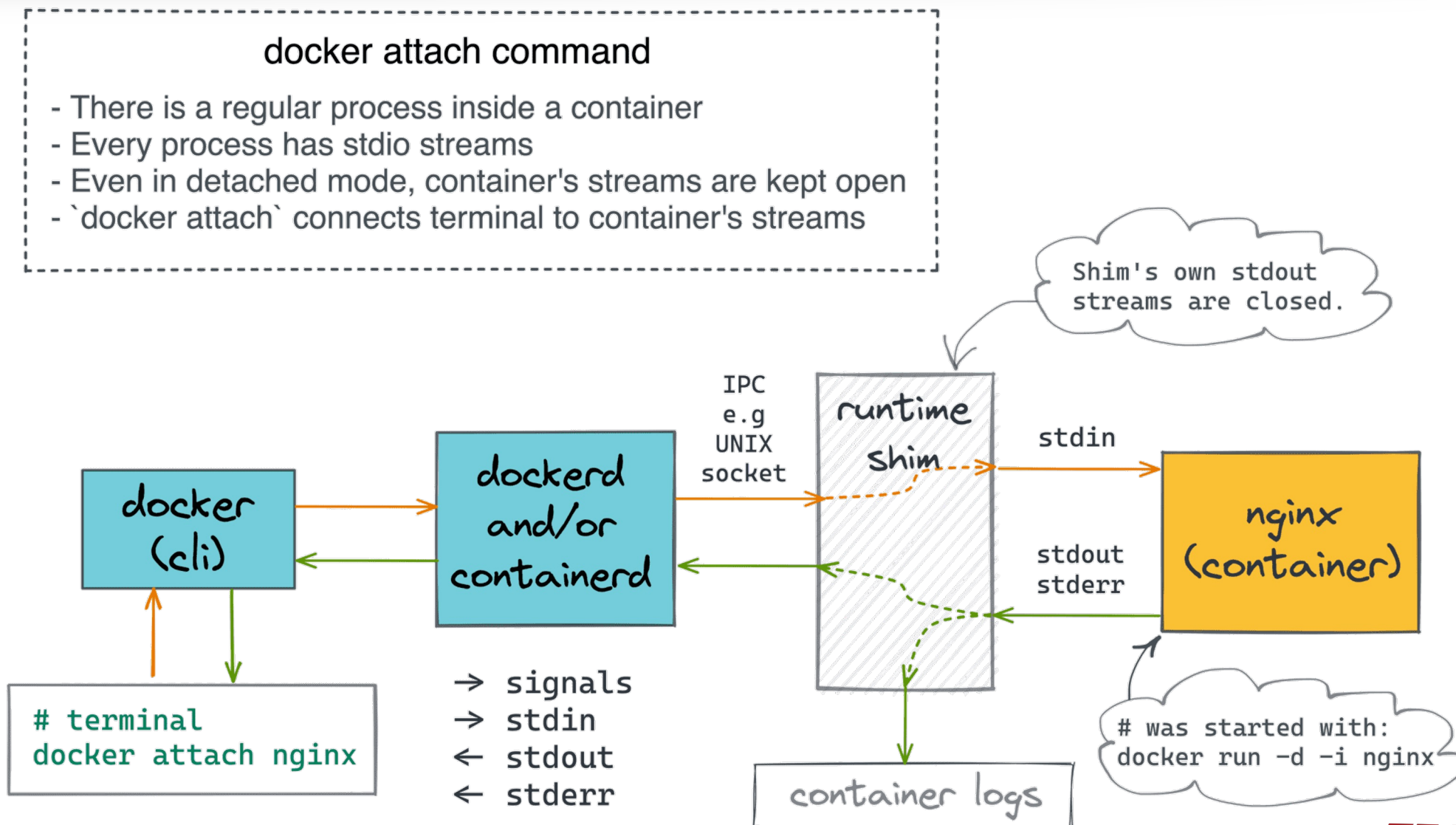




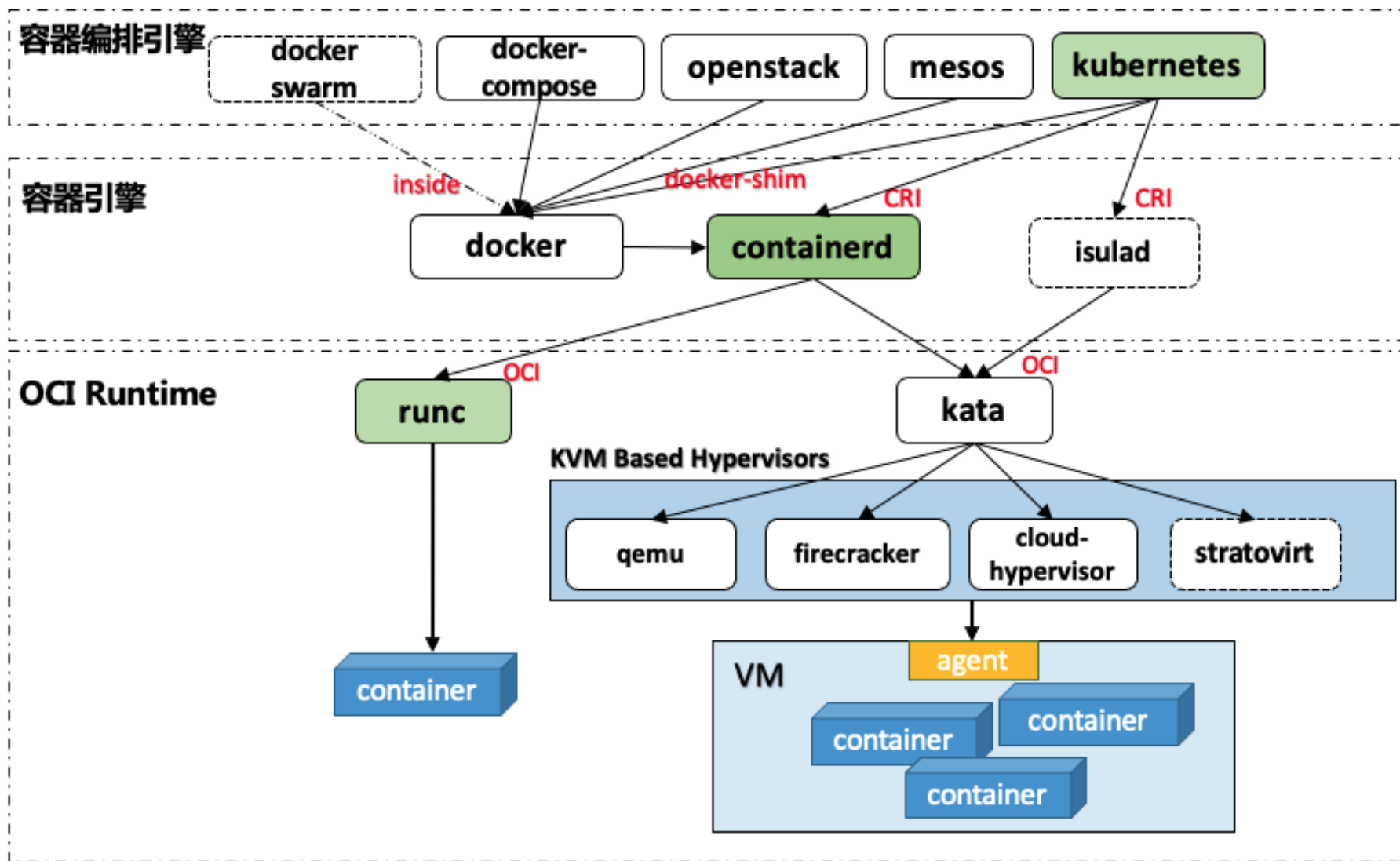
docker exec command

- OCI Runtime Spec does not define exec command
- Two containers can share namespaces and cgroups
- `docker exec` starts a temporary container reusing namespaces and cgroups of another container
- From the outside, it looks like running a command inside of this container

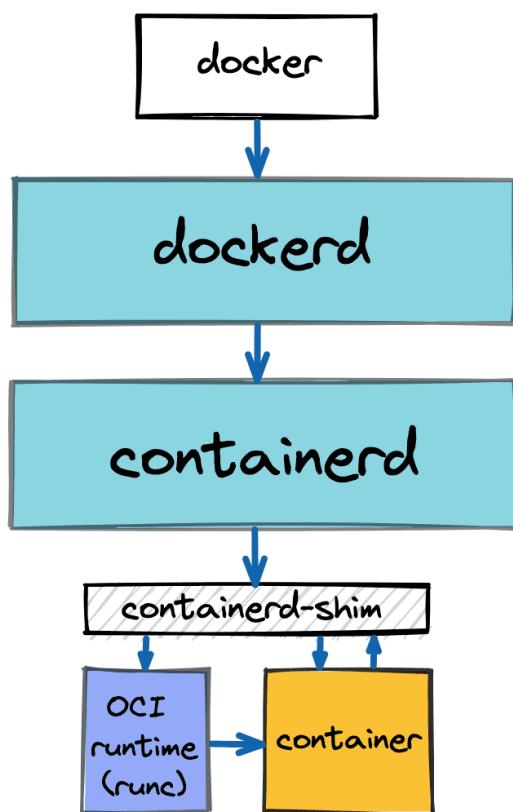




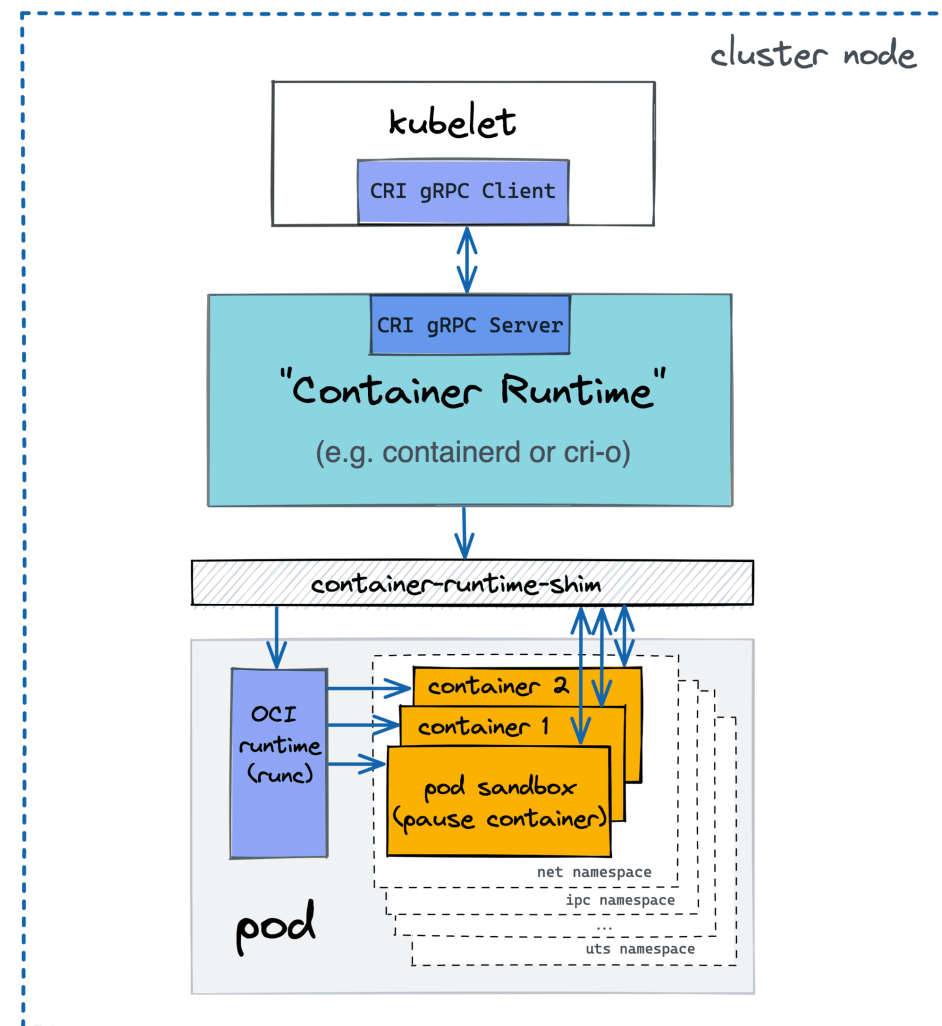
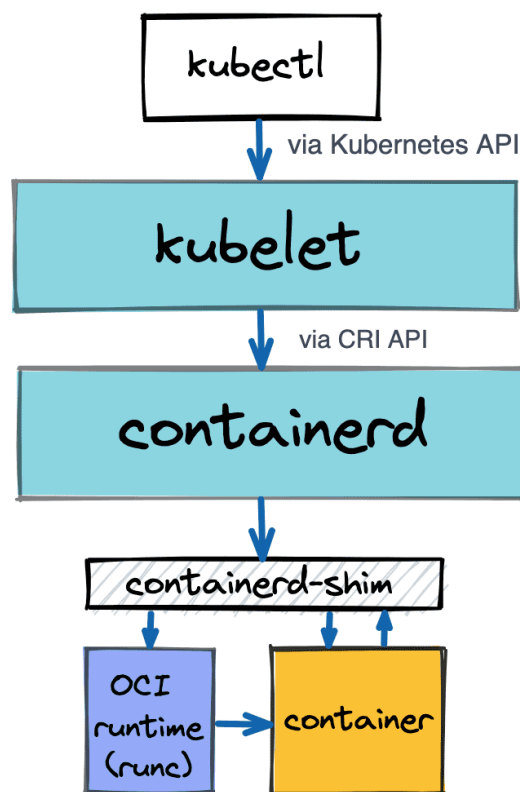
➤ Docker 及其构成组件在容器生态中的位置、角色



Docker



Kubernetes



➤ Namespace

- 基础概念

- **Namespace** 是一种 Linux 提供的一种内核级别环境（资源访问）隔离的机制，用来让运行在同一个操作系统上的进程之间资源隔离、互不干扰。

➤ Namespace

- 进一步理解
 - chroot 提供了一种简单的隔离模式，chroot 内部的文件系统无法访问外部的内容。
 - namespace 可以看做是一种加强版的 chroot，提供了对 UTS、IPC、Mount、PID、Network、User 等资源访问的隔离机制。在不同的 namespace 中无法看到其他 namespace 内部的资源。
- 通俗理解
 - 通过 namespace 可以让一些进程只能看到与自己相关的一部分资源，而另外一些进程也只能看到与它们自己相关的资源，这两拨进程就感觉不到对方的存在。通过系统调用构建一个相对隔离的 shell 环境。具体的实现方式是把一个或多个进程的相关资源指定在同一个 namespace 中。

➤ Namespace

- 如何实现一个资源隔离的容器

- 容器必然需要一个独立的 IP、端口、路由等网络资源 ----- **网络的隔离**
- 你的容器还需要一个独立的主机名以便在网络中标识自己 ----- **主机名及域名**
- 想到网络，顺其自然就想到通信，也就想到了进程间通信的隔离 ----- **信号量、消息队列和共享内存**
- 可能你也想到了权限的问题，对用户和用户组的隔离就实现了用户权限的隔离 ----- **用户和用户组**
- 文件系统（有哪些可用的文件系统，使用情况） ----- **文件系统隔离**
- 运行在容器中的应用需要有自己的 PID, 自然也需要与宿主机中的 PID 进行隔离 ----- **PID 的隔离**
- ...

- Namespace
 - 相关系统调用
 - clone() 实现线程的系统调用，用来创建一个新的进程，并可以通过设计上述参数达到隔离。
 - unshare() 使某进程脱离某个namespace
 - setns() 把某进程加入到某个namespace
 - 支持隔离的资源类型（ns）
 - Linux中支持多种系统资源的隔离，这里列出容器技术中用到的6种

Namespace	系统调用参数	隔离内容
UTS	CLONE_NEWUTS	主机名与域名
IPC	CLONE_NEWIPC	信号量、消息队列和共享内存
PID	CLONE_NEWPID	进程号
Network	CLONE_NEWNET	网络设备、网络栈、端口等等
Mount	CLONE_NEWNS	挂载点（文件系统）
User	CLONE_NEWUSER	用户和用户组

➤ Cgroups

- 基础概念

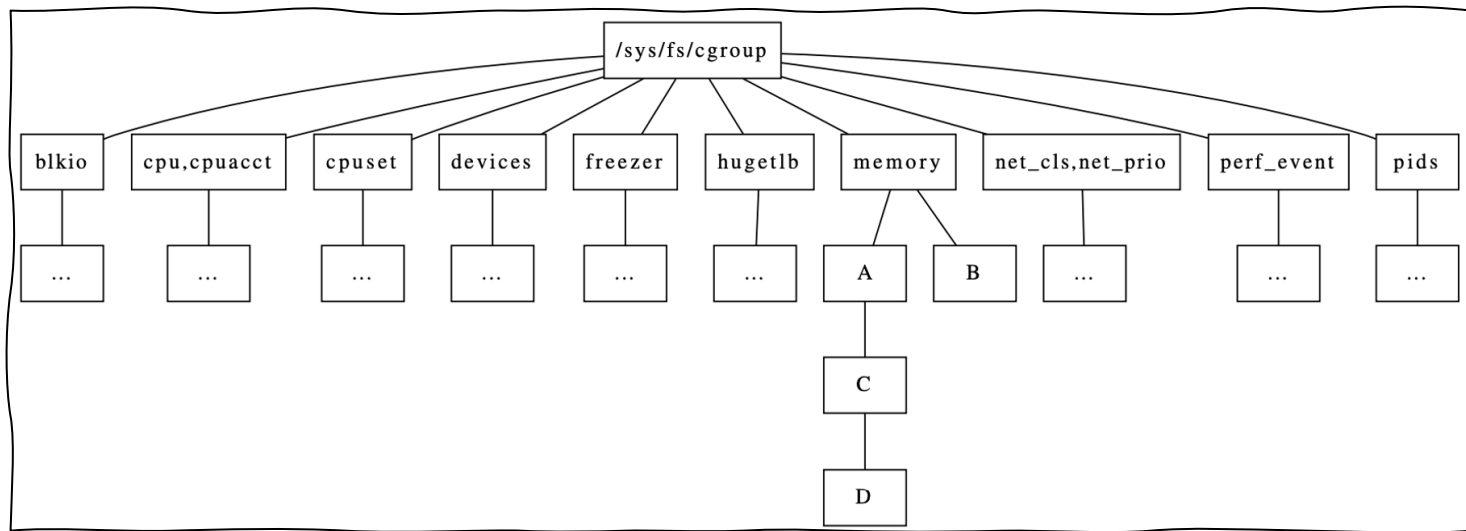
- Cgroups 是 Linux 内核提供的对一组进程做系统资源使用限制（资源QoS）的机制。
- 控制一组进程只能使用特定配额的资源，主要包括 CPU、内存、进程数、Block I/O 和网络带宽等。
- 由多个子控制器（Controller）构成，每个控制器控制一种资源

➤ Cgroups

- 在容器技术中使用 cgroup 来解决以下问题：
 - **限制**容器只能使用 2 个 CPU core
 - **拒绝**容器对某个特定的设备的访问
 - **统计**容器用了多少内存
 - **控制**容器可用（相比于另一个）双倍的 CPU 时间

➤ Cgroups

- Cgroup 中的控制器
 - 多个 controller 组成: cpu、mem、blkio、device ..
 - 每个 controller 控制一种资源
 - 每个 controller 都有各自独立的配置项
 - 层级化的结构, 子 cgroup 受到祖先 cgroup 限制
 - 资源可以在子 cgroup 中进一步划分



➤ Cgroups

- 使用接口
 - 没有 syscall 或 ioctl
 - 完全通过文件系统接口控制: 挂载、创建、读、写、删除。。
 - 以 cgroupfs 虚拟文件系统的方式暴露访问接口
 - 每种 controller 需要独立挂载
 - systemd 会自动挂载 cgroupfs 到/sys/fs/cgroup下

```
$ grep cgroup /proc/mounts
tmpfs /sys/fs/cgroup tmpfs ro,seclabel,nosuid,nodev,noexec,mode=755 0 0
cgroup /sys/fs/cgroup/systemd cgroup rw,seclabel,nosuid,nodev,noexec,relatime,xattr,name=systemd 0 0
cgroup /sys/fs/cgroup/cpu,cpuacct cgroup rw,seclabel,nosuid,nodev,noexec,relatime,cpu,cpuacct 0 0
cgroup /sys/fs/cgroup/memory cgroup rw,seclabel,nosuid,nodev,noexec,relatime,memory 0 0
cgroup /sys/fs/cgroup/cpuset cgroup rw,seclabel,nosuid,nodev,noexec,relatime,cpuset 0 0
cgroup /sys/fs/cgroup/freezer cgroup rw,seclabel,nosuid,nodev,noexec,relatime,freezer 0 0
cgroup /sys/fs/cgroup/net_cls,net_prio cgroup rw,seclabel,nosuid,nodev,noexec,relatime,net_cls,net_prio 0 0
cgroup /sys/fs/cgroup/perf_event cgroup rw,seclabel,nosuid,nodev,noexec,relatime,perf_event 0 0
cgroup /sys/fs/cgroup/blkio cgroup rw,seclabel,nosuid,nodev,noexec,relatime,blkio 0 0
cgroup /sys/fs/cgroup/pids cgroup rw,seclabel,nosuid,nodev,noexec,relatime,pids 0 0
cgroup /sys/fs/cgroup/hugetlb cgroup rw,seclabel,nosuid,nodev,noexec,relatime,hugetlb 0 0
cgroup2 /sys/fs/cgroup/unified cgroup2 rw,seclabel,nosuid,nodev,noexec,relatime,nsdelegate 0 0
```

- Cgroups
 - 文件接口操作

操作	用法
挂载某个cgroup controller	# mount -t cgroup -o memory memory /sys/fs/cgroup/memory
创建新的cgroup	# mkdir /sys/fs/cgroup/memory/new-cgroup
给某个cgroup设置限制	# echo 1073741824 > /sys/fs/cgroup/memory/new-cgroup/memory.limit_in_bytes
将某个进程加到某个cgroup中	# echo \$PID > /sys/fs/cgroup/memory/new-cgroup/cgroup.procs
读取cgroup中的某个stats	# cat /sys/fs/cgroup/memory/new-cgroup/memory.max_usage_in_bytes
读取cgroup中有哪些进程	# cat /sys/fs/cgroup/memory/new-cgroup/cgroup.procs
重命名某个cgroup	# mv /sys/fs/cgroup/memory/new-cgroup /sys/fs/cgroup/memory/different-name
移除某个cgroup (必须为空)	# rmdir /sys/fs/cgroup/memory/different-name

➤ Cgroups

- 文件接口操作
 - 一部分信息通过 procfs 的方式暴露出来
 - 查看某个进程所属的 cgroup

```
$ cat /proc/$PID/cgroup
11:hugetlb:/
10:freezer:/
9:memory:/user.slice/user-0.slice/session-1.scope
8:cpuset:/
7:devices:/user.slice
6:pids:/user.slice/user-0.slice/session-1.scope
5:blkio:/
4:net_cls,net_prio:/
3:perf_event:/
2:cpu,cpuacct:/
1:name=systemd:/user.slice/user-0.slice/session-1.scope
0::/user.slice/user-0.slice/session-1.scope
```

➤ Dockerfile、镜像与容器

- 三者之间的关系

- Dockerfile 是软件的原材料，Docker 镜像是软件的交付品，而 Docker 容器则可以认为是软件的运行态。
- 从应用软件的角度来看，Dockerfile、Docker 镜像与 Docker 容器分别代表软件的三个不同阶段。Dockerfile 面向开发，Docker 镜像成为交付标准，Docker 容器则涉及部署与运维，三者缺一不可，合力充当 Docker 体系的基石。

- 总结

- 容器是进程，镜像是程序包。
- 容器通过镜像来创建。
- 镜像是 Docker 生命周期的构建和打包阶段。
- 容器是 Docker 生命周期的启动和执行阶段。
- Dockerfile 是构建镜像的原材料。

➤ Docker 容器的本质

• 容器是一种沙盒技术

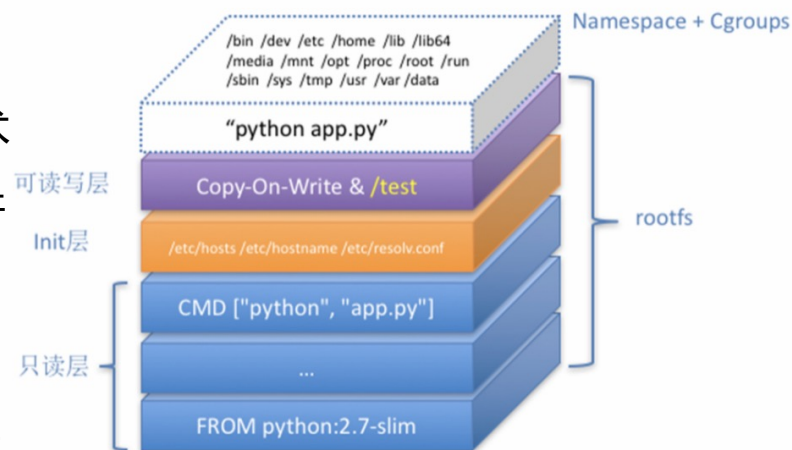
- 容器顾名思义，沙盒就是能够像一个集装箱一样，把应用“装”起来的技术。这样，应用自己运行在一个独立空间中；应用与应用之间，就因为有了边界而不至于相互干扰；而被装进集装箱的应用，也可以被方便地搬来搬去。

• 容器的本质是一种特殊的进程

- 这个进程是应用所有进程的父进程。容器技术通过环境隔离和资源限制，从而为进程创造出一个沙盒。Namespace 技术用来实现环境隔离，Cgroups 技术用来实现资源限制，rootfs 机制（也经常被称为容器镜像）为进程提供了文件系统。

• 创造沙盒比作造房子，把进程作为生活在屋子里的人

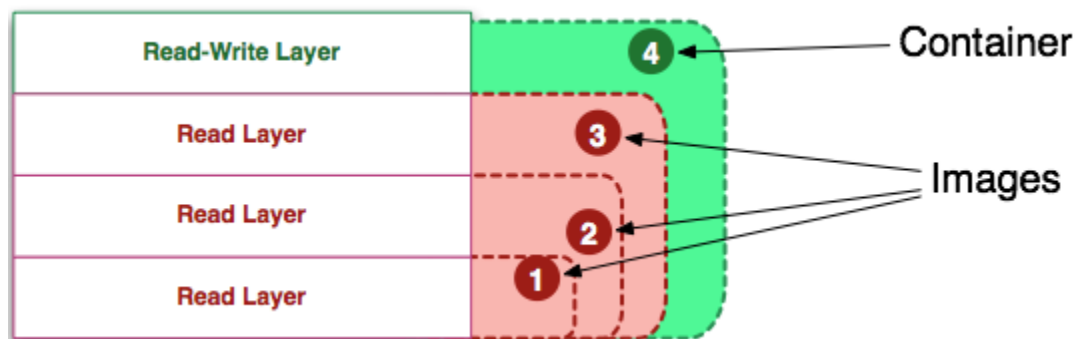
- Cgroups 规定了这个房子的面积，这个人能使用多少资源（CPU、内存、网络带宽等）。Namespace 建造了这个房子的四周墙壁和屋顶，把人与外界环境隔离开来。rootfs 机制建造了这个房子的地下室，存放着生活必需用品（应用所需文件，应用运行所需要的依赖和整个操作系统）。



➤ Docker 容器文件系统

- Docker 容器的 rootfs 使用的是 UnionFS（联合文件系统）。
- UnionFS 是通过联合挂载多个文件目录得到的一个完整的文件系统。容器 rootfs 看起来是一个可读、可写的具备完整功能的独立文件系统，但实际上由多层只读层和一个专属于该容器的读写层联合挂载而成。
- Linux 中支持的 UnionFS 实现有 overlay2、aufs、btrfs，其中最常用的就是 overlay2。

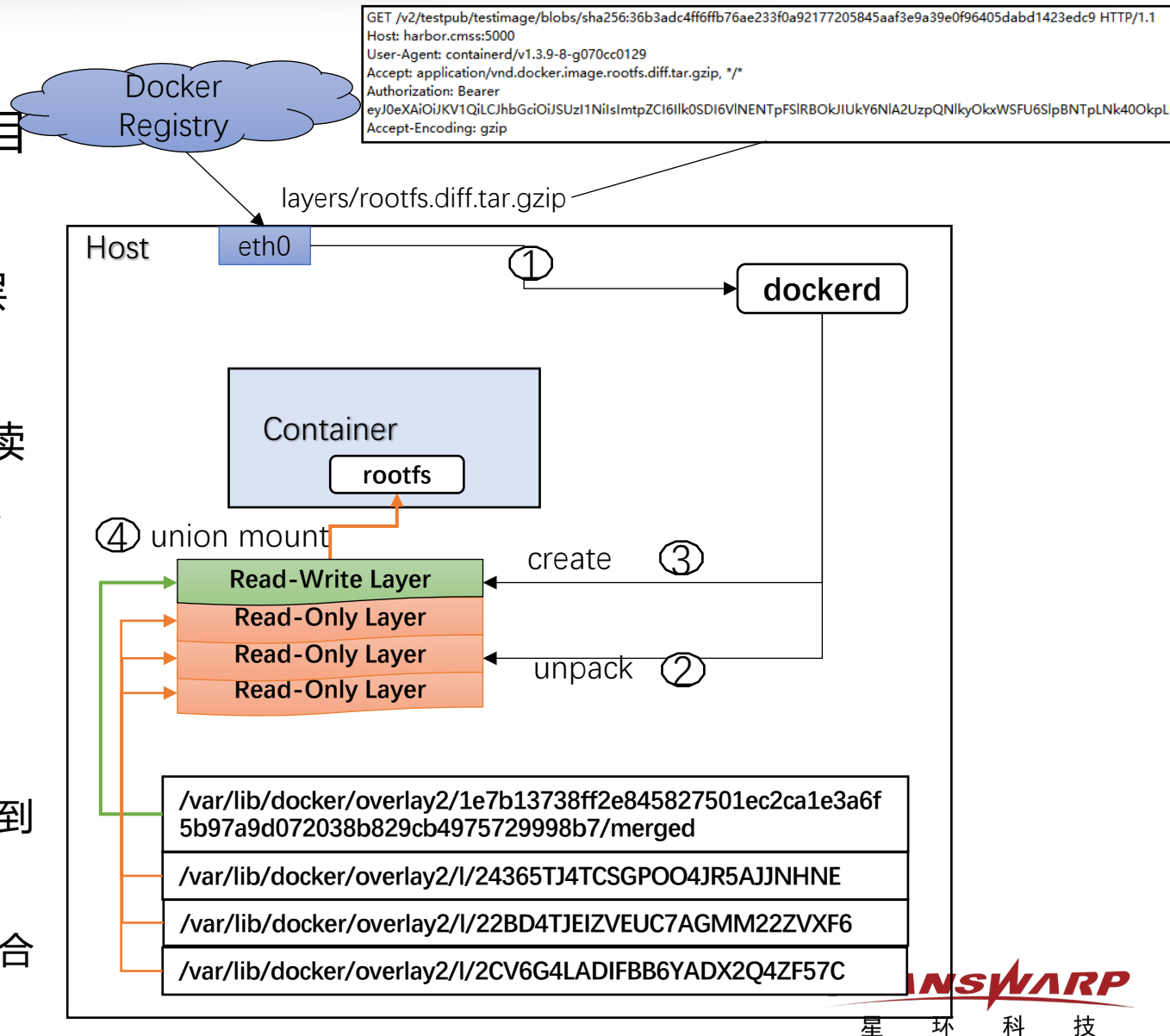
```
$ sudo docker info
...
Server Version: 20.10.12
Storage Driver: overlay2
Backing Filesystem: extfs
Supports d_type: true
Native Overlay Diff: true
userxattr: false
...
```



Driver	Description	Supported backing filesystems
overlay2	overlay2 是所有当前支持的 Linux 发行版的首选存储驱动程序，不需要额外配置	xfs with ftype=1, ext4
fuse-overlayfs	fuse-overlayfs 仅适用于在不支持 rootless overlay2 的主机上运行 Rootless Docker。在 Ubuntu 和 Debian 10 上，不需要使用 fuse-overlayfs 驱动程序，overlay2 即使在 rootless 下也可以工作。	any filesystem
btrfs and zfs	btrfs 和 zfs 存储驱动程序允许高级选项，例如创建“快照”，但需要更多的维护和设置。其中每一个都依赖于正确配置的后台文件系统	btrfs / zfs
vfs	vfs 存储驱动程序旨在用于测试目的，以及无法使用写时复制文件系统的情况。VFS 性能很差，一般不建议用于生产	any filesystem
devicemapper	devicemapper 存储驱动程序在生产环境中需要使用 direct-lvm，因为 loopback-lvm 虽然是零配置，但性能非常差。devicemapper 是 CentOS 和 RHEL 推荐的存储驱动程序，因为它们之前的内核版本不支持 overlay2。但当前版本的 CentOS 和 RHEL 都已支持 overlay2，这是现在推荐的驱动程序	direct-lvm

➤ Docker 容器文件系统

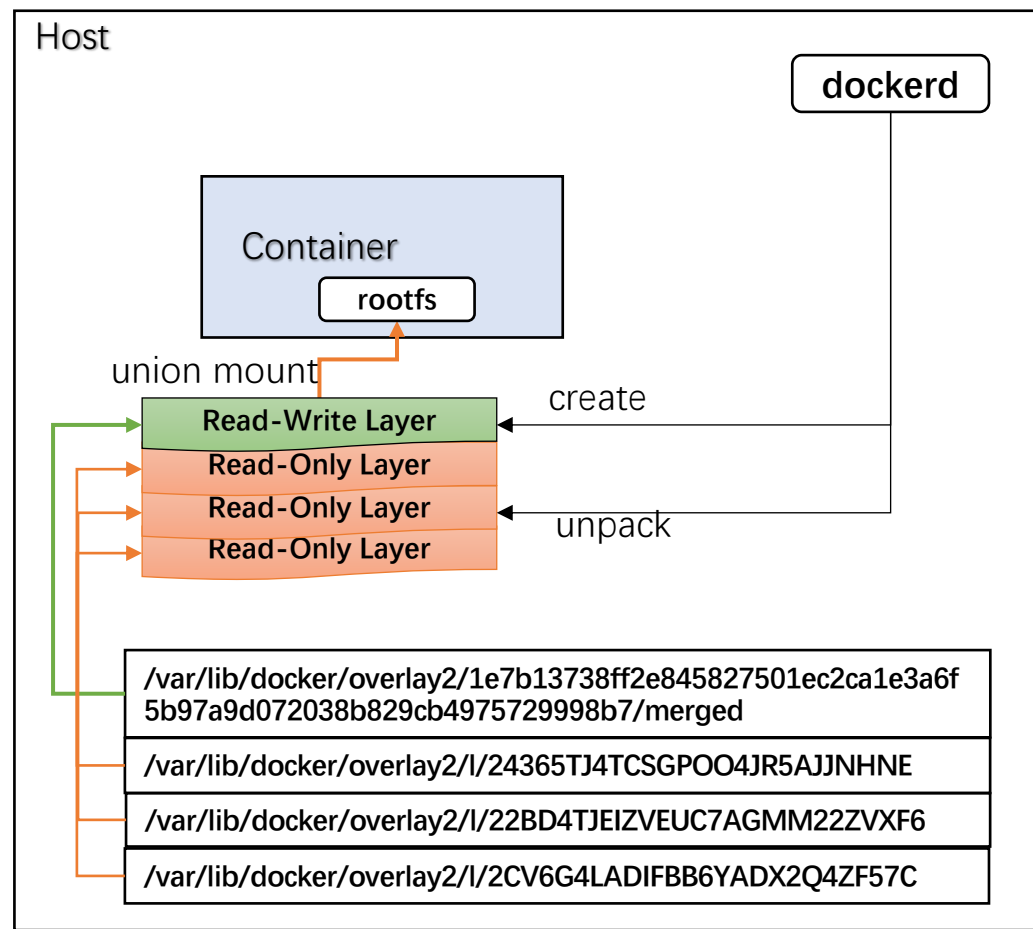
- 在准备容器 rootfs 时联合挂载用到的多层目录是哪来的呢？
 - 读写层**，每个容器都有自己专属的读写层，该层由 docker 在创建此容器时创建的。
 - 只读层**，每个容器的 rootfs 都会包含好几层只读层，该层由 docker 在拉取镜像时将镜像中的各层解压到 docker 的数据目录得到的。
- 总结
 - 容器镜像与容器 rootfs 的关联
 - ✓ Docker 拉取镜像解压到 docker 数据目录后得到只读层；
 - ✓ Docker 创建容器时使用数据目录中的读写层联合挂载得到容器 rootfs



➤ Docker 容器文件系统

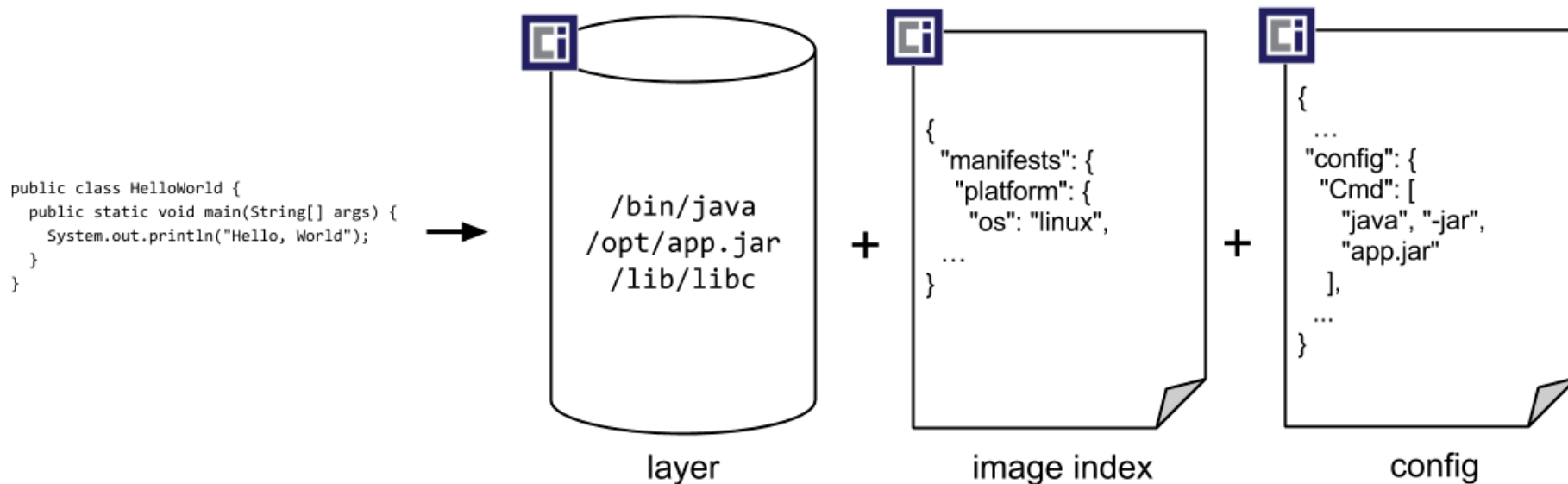
- 模拟 Docker 准备容器 rootfs 的过程
 - 使用如下脚本通过手动执行联合挂载，模拟 Docker 准备容器 rootfs

```
mkdir -p /path/to/ro-image-layer1
mkdir -p /path/to/ro-image-layer2
mkdir -p /path/to/rw-container-layer
mkdir -p /path/to/container-rootfs
mkdir -p /tmp/work
mount -t overlay overlay -o rw,lowerdir=/path/to/ro-image-
layer1:/path/to/ro-image-layer2,upperdir=/path/to/rw-container-
layer,workdir=/tmp/work /path/to/container-rootfs
```



➤ Docker 容器镜像格式

- 一个镜像由 manifest、image index、filesystem layers 和 configuration 四部分组成
 - Image Index 和 manifest 的关系是一对多
 - Image manifest 和 config 的关系是一对一的关系
 - Image manifest 和 Filesystem layers 是一对多的关系



➤ Docker 容器镜像格式

- filesystem layers

- Filesystem Layer 包含了文件系统的信息
- 描述 image 包含了哪些文件/目录, 以及它们的属性和数据
- 包含的内容, 每个 filesystem layer 都包含在上一个 layer 上的改动情况, 主要包括 3 个方面:
 - ✓ **变化类型**: 是增加、修改还是删除了文件
 - ✓ **文件类型**: 每个变化发生在哪种文件类型上
 - ✓ **文件属性**: 文件的修改时间、用户ID、组ID、RWX权限等 比如在某一层增加了一个文件, 那么这一层所包含的内容就是增加的这个文件的数据以及它的属性
- 打包格式
 - ✓ 最终每个layer都会打包成一个文件, 这个文件的格式可以是 tar 和 tar + gzip 两种中的一种。

➤ Docker 容器镜像格式

- image config

- image config 就是一个json文件，它的 media type 是 application/vnd.oci.image.config.v1+json，这个 json 文件包含了对这个 image 的描述

- 重要属性

- ✓ architecture: CPU 架构，例如：amd64、arm64
 - ✓ os: 操作系统，例如：linux
 - ✓ config: 当使用此image启动container时，config中的配置就是运行container时的默认参数
 - ✓ rootfs: 指定了 image 所包含的 filesystem layers，type 的值必须是 layers，diff_ids 包含了 layer 的列表（顺序排列），每一个 sha256 就是每层 layer 对应 tar 包的 sha256

```
{
  "created": "2015-10-31T22:22:56.015925234Z",
  "author": "Alyssa P. Hacker <alyspdev@example.com>",
  "architecture": "amd64",
  "os": "linux",
  "config": {
    "User": "alice",
    "ExposedPorts": {
      "8080/tcp": {}
    },
    "Env": [
      "PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin",
      "FOO=oci_is_a",
      "BAR=well_written_spec"
    ],
    "Entrypoint": [
      "/bin/my-app-binary"
    ],
    "Cmd": [
      "--foreground",
      "--config",
      "/etc/my-app.d/default.cfg"
    ],
    "Volumes": {
      "/var/job-result-data": {},
      "/var/log/my-app-logs": {}
    },
    "WorkingDir": "/home/alice",
    "Labels": {
      "com.example.project.git.url": "https://example.com/project.git",
      "com.example.project.git.commit": "45a939b2999782a3f005621a8d0f29aa387e1d6b"
    }
  },
  "rootfs": {
    "diff_ids": [
      "sha256:c6f988f4874bb0add23a778f753c65efe992244e148a1d2ec2a8b664fb66bbd1",
      "sha256:5f70bf18a086007016e948b04aed3b82103a36bea41755b6cddfaf10ace3c6ef"
    ],
    "type": "layers"
  },
}
```

➤ Docker 容器镜像格式

- manifest

- Manifest 也是一个 json 文件, media type 为 application/vnd.oci.image.manifest.v1+json, 这个文件包含了对前面 filesystem layers 和 image config 的描述
 - ✓ Config 里面包含了对 image config 文件的描述, 有 media type, 文件大小, 以及 sha256
 - ✓ Layers 包含了对每一个 layer 的描述, 和对 config 文件的描述一样, 也包含了 media type, 文件大小, 以及 sha256

```
{
  "schemaVersion": 2,
  "config": {
    "mediaType": "application/vnd.oci.image.config.v1+json",
    "size": 7023,
    "digest": "sha256:b5b2b2c507a0944348e0303114d8d93aaaa081732b86451d9bce1f432a537bc7"
  },
  "layers": [
    {
      "mediaType": "application/vnd.oci.image.layer.v1.tar+gzip",
      "size": 32654,
      "digest": "sha256:9834876dcfb05cb167a5c24953eba58c4ac89b1adf57f28f2f9d09af107ee8f0"
    },
    {
      "mediaType": "application/vnd.oci.image.layer.v1.tar+gzip",
      "size": 16724,
      "digest": "sha256:3c3a4604a545cdc127456d94e421cd355bca5b528f4a9c1905b15da2eb4a4c6b"
    },
    {
      "mediaType": "application/vnd.oci.image.layer.v1.tar+gzip",
      "size": 73109,
      "digest": "sha256:ec4b8955958665577945c89419d1af06b5f7636b4ac3da7f12184802ad867736"
    }
  ],
  "annotations": {
    "com.example.key1": "value1",
    "com.example.key2": "value2"
  }
}
```

➤ Docker 容器镜像 layout

- image layout
 - image layout 是描述镜像从 registry 拉到本地，以什么样的目录结构保存的。
 - image layout 目前主要有两种标准：docker-layout 和 oci-ayout

➤ Docker 容器镜像 layout

- docker-layout: docker 定义的镜像存储 Layout

```
root@transwarp:~# docker save nginx -o nginx.tar
root@transwarp:~# tar -tf nginx.tar
0cb59b23812a64ae9115a5cf081f05d3d1915c5d7d2304c64b1c3a07629122a9/
0cb59b23812a64ae9115a5cf081f05d3d1915c5d7d2304c64b1c3a07629122a9/VERSION
0cb59b23812a64ae9115a5cf081f05d3d1915c5d7d2304c64b1c3a07629122a9/json
0cb59b23812a64ae9115a5cf081f05d3d1915c5d7d2304c64b1c3a07629122a9/layer.tar
0e9f59de25dca39e1cc33d1b7a83f27668c800221d7f1062a5da0df9ba684f44/
0e9f59de25dca39e1cc33d1b7a83f27668c800221d7f1062a5da0df9ba684f44/VERSION
0e9f59de25dca39e1cc33d1b7a83f27668c800221d7f1062a5da0df9ba684f44/json
0e9f59de25dca39e1cc33d1b7a83f27668c800221d7f1062a5da0df9ba684f44/layer.tar
5df3f744c961b85bb93947d6a4288543c06e64b3130562ae7be85fdc9f7a5691/
5df3f744c961b85bb93947d6a4288543c06e64b3130562ae7be85fdc9f7a5691/VERSION
5df3f744c961b85bb93947d6a4288543c06e64b3130562ae7be85fdc9f7a5691/json
5df3f744c961b85bb93947d6a4288543c06e64b3130562ae7be85fdc9f7a5691/layer.tar
605c77e624ddb75e6110f997c58876baa13f8754486b461117934b24a9dc3a85.json
728205feeb04d15cbf0b889a9336e553bf6d472a311724e567a156afe3cb105d/
728205feeb04d15cbf0b889a9336e553bf6d472a311724e567a156afe3cb105d/VERSION
728205feeb04d15cbf0b889a9336e553bf6d472a311724e567a156afe3cb105d/json
728205feeb04d15cbf0b889a9336e553bf6d472a311724e567a156afe3cb105d/layer.tar
7e8578b7bc47a5f07742264aace8ee761078c30c1c2c2dafbc79a8ebd340c191/
7e8578b7bc47a5f07742264aace8ee761078c30c1c2c2dafbc79a8ebd340c191/VERSION
7e8578b7bc47a5f07742264aace8ee761078c30c1c2c2dafbc79a8ebd340c191/json
7e8578b7bc47a5f07742264aace8ee761078c30c1c2c2dafbc79a8ebd340c191/layer.tar
8c46511b06be8a5ed3d1478039cd8e3bcb0842b8a68c42babb5e9c4b6dc6beeb/
8c46511b06be8a5ed3d1478039cd8e3bcb0842b8a68c42babb5e9c4b6dc6beeb/VERSION
8c46511b06be8a5ed3d1478039cd8e3bcb0842b8a68c42babb5e9c4b6dc6beeb/json
8c46511b06be8a5ed3d1478039cd8e3bcb0842b8a68c42babb5e9c4b6dc6beeb/layer.tar
manifest.json
repositories
```

➤ Docker 容器镜像 layout

- oci-layout: OCI 标准定义的镜像存储 Layout

```
[root@transwarp:~# nerdctl save nginx -o nginx-oci.tar
[root@transwarp:~# tar -tf nginx-oci.tar
blobs/
blobs/sha256/
blobs/sha256/091c283c6a66ad0edd2ab84cb10edacc00a1a7bc5277f5365c0d5c5457a75aff
blobs/sha256/1ae07ab881bd848493ad54c2ba32017f94d1d8dbfd0ba41b618f17e80f834a0f
blobs/sha256/2834dc507516af02784808c5f48b7cbe38b8ed5d0f4837f16e78d00deb7e7767
blobs/sha256/55de5851019b8f65ed6e28120c6300e35e556689d021e4b3411c7f4e90a9704b
blobs/sha256/5eb5b503b37671af16371272f9c5313a3e82f1d0756e14506704489ad9900803
blobs/sha256/78091884b7bea0fa918527207924e9993bcc21bf7f1c9687da40042ceca31ac9
blobs/sha256/b559bad762bec166fd028483dd2a03f086d363ee827d8c98b7268112c508665a
blobs/sha256/bb129a712c2431ecce4af8dde831e980373b26368233ef0f3b2bae9e9ec515ee
blobs/sha256/c316d5a335a5cf324b0dc83b3da82d7608724769f6454f6d9a621f3ec2534a5a
index.json
manifest.json
oci-layout
```

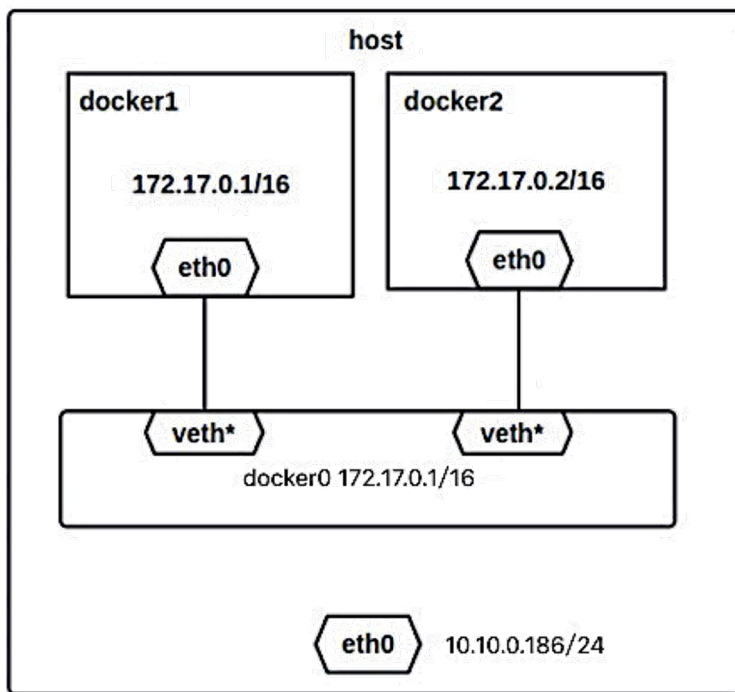
➤ Docker 网络

当安装 Docker 时，它会自动创建三个网络：host, bridge 和 none。创建容器时，可以使用该--network标志来指定容器应连接到哪些网络。

网络模式	简介
Host	容器将不会虚拟出自己的网卡，配置自己的 IP 等，而是使用宿主机的 IP 和端口。
Bridge	此模式会为每一个容器分配、设置 IP 等，并将容器连接到一个 docker0 虚拟网桥，通过 docker0 网桥以及 iptables nat 表配置与宿主机通信。
None	该模式关闭了容器的网络功能。
Container	创建的容器不会创建自己的网卡，配置自己的 IP，而是和一个指定的容器共享 IP、端口范围。
自定义网络	macvlan、ipvlan 等

➤ Docker 网络 – Bridge 模式

- 在 Docker Server 启动时，默认会在主机上创建一个名为 docker0 的虚拟网桥，该主机上所有启动的 Docker 容器会连接到这个虚拟网桥上，其工作方式类似于物理交换机，主机上的所有容器就通过交换机连在了一个二层网络中。
- 接着要为容器分配 IP 了，默认会从私有 IP 网段中，选择一个和宿主机不同的 IP 地址和子网分配给 docker0，所有连接到 docker0 的容器就从这个子网中分配一个未占用的 IP。



如上容器中，使用的是 172.17.0.0/16 这个网段，并将 172.17.0.1 分配给 docker0 网桥。

4 chapter

容器技术标准

✓ OCI (Open Container Initiative)

➤ OCI 概念

• Open Container Initiative 开发容器标准

- 早期 Docker 的实现或者说发展方向主要是由 Docker公司控制的，并没有一个统一的工业标准。这对于一些头部公司而言，显然是不能接受的，没有统一的工业标准意味着如果选择了使用 Docker 的容器化技术，便会被 Docker 公司所绑定；加上随着 Docker 软件的升级，某些功能或者特性必然会进行变动，没人能保证不发生破坏性变更。为了推进容器化技术的工业化，2015 年 6 月在 DockerCon 上 Docker、CoreOS、Google、RedHat 等公司共同宣布成立开放容器项目 OCI，Docker 公司将 libcontainer 捐出并改名 runC，交由中立的 Linux 基金会管理，并以 runC 为依据，共同制定一套容器和镜像的标准和规范。
- libcontainer: Docker 公司发布的容器运行时库，runC 的前身。
- OCI 规范实际上将容器运行时和容器镜像从 Docker 项目中剥离形成的容器技术标准。

- OCI 的目标
 - 建立容器格式和运行时的工业开放通用标准
- OCI 制定的容器规范
 - runtime-spec 容器运行时规范
 - image-spec 容器镜像规范
 - distribution-spec 镜像分发规范
- OCI 规范之间的联系
 - runtime-spec 与 image-spec 通过 OCI runtime filesystem bundle 的标准格式关联
 - OCI 镜像 (image-spec) 能够经过工具转换成 bundle
 - OCI 容器引擎 (runtime-spec) 使用这个 bundle 来运行容器
 - OCI Registry (distribution-spec) 负责存储和分发OCI镜像 (image-spec)

➤ Image Spec

- 文件系统：以 layer 保存的文件系统，每一个 layer 保存了和上层之间变化的部分，layer 应该保存哪些文件，怎么表示增长、修改和删除的文件等接口
- config 文件：保存了文件系统的层级信息（每一个层级的 hash 值，以及历史信息），以及容器运行时须要的一些信息（环境变量、工作目录、命令参数、mount 列表），指定了镜像在某个特定平台和系统的配置。比较 docker inspect <image-id> 看到的内容
- manifest 文件：镜像的 config 文件索引，有哪些 layer，额外的 annotation 信息，manifest 文件中保存了和当前平台相关的信息
- index 文件：可选的文件，指向不一样平台的 manifest 文件，该文件用于保证一个镜像能够跨平台使用，每一个平台拥有不一样的 manifest 文件，使用 index 做为索引

➤ Runtime Spec

- creating: 使用 create 命令建立容器, 这个过程称为创建中
- created: 容器建立出来, 但未运行, 表示镜像和配置没有错误, 容器可以运行在当前平台
- running: 容器的运行状态, 里面的进程处于 up 状态, 正在执行用户设定的任务
- stopped: 容器运行完成, 或者运行出错, 或者 stop 命令以后, 容器处于暂停状态。这个状态, 容器还有不少信息保存在平台中, 并无彻底被删除

➤ 支持OCI标准的实现

- 容器运行时
 - runc
 - kata-container
 - gVisor
- 容器生命周期管理
 - containerd
 - docker
 - pod-man
 - cri-o
- 容器镜像构建
 - buildkit
 - buildah
 - skopeo (容器镜像格式转换工具)

➤ OCI Image 与 Docker Image

- OCI 的建立推动了容器技术的工业化，但是否此标准就是唯一呢？其实不然。在成立 OCI 并制定 image-spec 标准的时候 Docker 已经空前繁荣，并得到了广泛的应用。由于标准只定义了最基本的内容，想要将 Docker 的实现全部按照标准进行改造的话，会对 Docker 造成破坏性变更，也不利于 Docker 功能的迭代。Docker 为了支持 OCI 标准的普及，已经推进了 registry 对 OCI 镜像的支持，现在也正在给 Docker 自身增加适配中，目标是让 Docker 支持两种镜像格式，分别是符合 Docker 标准的镜像和符合 OCI 标准的镜像。



Q&A

TRANSWARP
星 环 科 技

温故知新

- 思考题1：构建一个 2048 游戏的镜像
- 思考题2：使用容器方式运行 Nginx，并使用本地挂载对外提供的网页
- 思考题3：容器文件系统与挂载的区别
- 课后题：[2023 - 基于云原生技术的软件开发 - 第一次作业](#)

